

Citation for published version:

Erdoğan, G & Yildirim, EA 2021, 'Exact and Heuristic Algorithms for the Carrier-Vehicle Traveling Salesman Problem', *Transportation Science*, vol. 55, no. 1, pp. 101-121. <https://doi.org/10.1287/trsc.2020.0999>

DOI:

[10.1287/trsc.2020.0999](https://doi.org/10.1287/trsc.2020.0999)

Publication date:

2021

Document Version

Peer reviewed version

[Link to publication](#)

Publisher Rights

CC BY-NC

University of Bath

Alternative formats

If you require this document in an alternative format, please contact:
openaccess@bath.ac.uk

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

Take down policy

If you believe that this document breaches copyright please contact us providing details, and we will remove access to the work immediately and investigate your claim.

Authors are encouraged to submit new papers to INFORMS journals by means of a style file template, which includes the journal title. However, use of a template does not certify that the paper has been accepted for publication in the named journal. INFORMS journal templates are for the exclusive purpose of submitting to an INFORMS journal and should not be used to distribute the papers in print or online or to submit the papers to another publication.

Exact and Heuristic Algorithms for the Carrier-Vehicle Traveling Salesman Problem

Güneş Erdoğan

School of Management, University of Bath, Claverton Down, Bath, BA2 7AY, United Kingdom, G.Erdogan@bath.ac.uk

E. Alper Yıldırım

School of Mathematics, The University of Edinburgh, Peter Guthrie Tait Road, Edinburgh, EH9 3FD, United Kingdom,
E.A.Yildirim@ed.ac.uk

This paper presents new structural properties for the Carrier-Vehicle Traveling Salesman Problem. The authors provide a new mixed integer second order conic optimization formulation, with associated optimality cuts based on the structural properties, and an Iterated Local Search (ILS) algorithm. Computational experiments on instances from the literature demonstrate the superiority of the new formulation to the existing models and algorithms in the literature, and the high quality solutions found by the ILS algorithm.

Key words: traveling salesman problem; multi-vehicle systems; mixed integer second order conic optimization

April 25, 2020

1. Introduction

Several complex mission planning operations necessitate the coordination and cooperation of heterogeneous vehicles with different but complementary capabilities (see, e.g., Murray (2007)). For instance, the recent technological advances in the capabilities of unmanned aerial vehicles (UAVs), also known as drones, facilitate their use together with other ground vehicles or naval vessels in order to improve the effectiveness, speed, range, safety, and cost of operations in various humanitarian, ecological, environmental, and military applications.

In this paper, we focus on a two-vehicle system consisting of a slow but large vehicle, referred to as the Carrier (e.g., a ship), with a virtually unlimited operational capability, and a faster but smaller vehicle, referred to as the Vehicle, with a limited operational capability (e.g., a helicopter

or a drone). The Carrier is capable of transporting, deploying, recovering, and servicing the Vehicle. Such systems are frequently used in search-and-rescue, surveillance, monitoring, and logistics operations. In each such operation, there is a set of target points to be visited by the Vehicle for the purpose of rescuing victims, taking pictures and certain measurements, or performing delivery and pick-up operations. For each target point, the Vehicle should leave the Carrier at some take-off point, visit the target point, and return to the Carrier for servicing, refueling or recharging purposes without exhausting its limited operational capability. We focus on the problem of planning and coordinating the routes of the Carrier and the Vehicle in such a way that all of the target points are visited in minimum total time starting from an origin and ending at a prespecified destination. This problem, called the Carrier-Vehicle Traveling Salesman Problem (CVTSP), has been introduced by Garone et al. (2010b).

The CVTSP is applicable in settings in which the Carrier can travel, without any restriction, in the Euclidean plane. With the rapid advances in the capabilities of drones, the CVTSP finds interesting applications in maritime search-and-rescue operations, mapping oil spills in the oceans, monitoring operations in marine sciences, offshore platform logistics, and, more recently, in military operations in which drones are launched from and recovered by a plane. Due to the different capabilities of the Vehicle and the Carrier, there are several settings in which the Carrier may not necessarily have the capability required during the visit of a target point, as in surveillance, monitoring, or offshore logistics operations. Alternatively, it may be impractical, costly, or even risky for the Carrier to visit a target point, as in search-and-rescue and military operations. The CVTSP is particularly applicable in time-critical marine based search-and-rescue operations as well as post-disaster operations after marine oil spills. Any improvement of the planning process would translate into more lives saved, reduced environmental damage, and decreased cost of operations.

Solving the CVTSP requires the determination of the sequence of the target points to be visited, which is inherently a combinatorial problem, and the computation of the take-off and landing points for each target point, which is a continuous problem. Indeed, under the assumption that the Vehicle and Carrier speeds are identical, the CVTSP reduces to the minimum-cost Hamiltonian path problem, or the Euclidean Traveling Salesman Problem (TSP) if the origin and destination are identical. Therefore, the CVTSP is, in general, NP-hard.

We next briefly review the literature on the CVTSP. A simpler version of the CVTSP has been introduced in Garone et al. (2008), where the order of the target points to be visited is a priori fixed, referred to as the Carrier Vehicle Problem (CVP). The authors obtain closed-form solutions for some special cases with up to two target points. Building on the results from these simpler cases, they propose a heuristic for an arbitrary number of target points. The more general version of the CVTSP, in which the order of the target points is not a priori given, is studied

in Garone et al. (2010b, 2011). The authors observe that the CVP can be formulated as a convex optimization problem. They propose a two-stage heuristic method, in which a near-optimal solution to the Euclidean Traveling Salesman Problem on the target points is computed in the first stage by employing a polynomial-time approximation scheme and the CVP is solved by fixing this order in the second stage. For the resulting solution, they establish a worst-case approximation error bound, which depends on the approximation error in the first stage as well as some data-dependent parameters. In Garone et al. (2011), the authors propose enumerating all possible sequences of target points and solving the CVTSP with each fixed sequence for instances with up to 5 target points.

In Gambella et al. (2018), an exact solution approach based on a mixed integer second order conic (MISOC) optimization model is proposed for the CVTSP. The main algorithmic contribution of the authors is a Ranking Based Algorithm (RBA) that is based on enumeration of Hamiltonian paths starting from the origin, visiting the set of target points, and ending at the destination. For a fixed Hamiltonian path, they compute a lower bound on the optimal value of the CVTSP. The paths are then ranked in ascending order of the lower bounds. For each path in that order, they solve the CVP with this fixed sequence to obtain an upper bound. The procedure terminates with an optimal solution if the upper bound matches the lower bound. The authors report optimal solutions on instances with up to 15 target points in less than an hour of CPU time.

A further extension of the CVTSP, referred to as the Generalized Carrier-Vehicle Traveling Salesman Problem (GCVTSP), is studied in Garone et al. (2014), where the Vehicle is allowed to visit multiple target points between a take-off and landing. A mixed integer nonlinear optimization model is presented and a three-stage heuristic method is proposed, which is composed of approximately solving the Euclidean TSP on the target points in order to fix the sequence in the first stage, determining the cluster of target points to be visited between each take-off and landing for the given sequence in the second stage, and computing the take-off and landing points for each cluster in the third stage. In Klauco et al. (2014), the authors formulate the GCVTSP with a fixed order of target points as a mixed integer second order conic optimization problem and report exact solutions on instances with 30 – 100 target points in $10^3 - 10^5$ seconds.

After the initial version of this manuscript was submitted, we became aware of a related study on the Mothership and Drone Routing Problem introduced by Poikonen and Golden (2019), which is essentially the same problem as the CVTSP studied in this paper. The authors propose an exact branch-and-bound method, which is based on systematically enumerating all the visit sequences and solving the corresponding CVP to compute the take-off and landing points for each fixed sequence. They also propose several heuristic methods. Their exact method is able to solve randomly generated instances with up to 20 target points. They also consider the extension in which the Vehicle is allowed to visit multiple target points.

We note that the CVTSP is defined in the Euclidean plane, and there is a separate extensive literature on multi-vehicle systems defined on networks. One of the earliest such problems is the Flying Sidekick Traveling Salesman Problem introduced by Murray and Chu (2015), inspired by the drone delivery systems of logistics companies and e-tailers. Similar problems with slight variations have been studied (see, e.g., Agatz et al. (2018), Bouman et al. (2018), Ha et al. (2018), Saleu et al. (2018), Poikonen et al. (2019)). For other variants of the problem including multiple vehicles and multiple drones, we refer the reader to Murray and Raj (2020) and the references therein. In each of these variants of the problem, the landing and take-off points of the Vehicle are restricted to the set of the locations of target points or the depot (i.e., the origin in the context of the CVTSP). This assumption makes the problem amenable to a mixed integer linear programming formulation. In contrast, since the Vehicle is allowed to take-off and land at any point in the plane in the CVTSP, the synchronization of the Carrier and the Vehicle necessitates the use of nonlinear Euclidean distance constraints. As such, the CVTSP is a fundamentally different problem and its exact formulation requires a mixed integer nonlinear model (see Section 3).

Another related problem is the Traveling Salesman Problem with Neighborhoods (TSPN), introduced in Arkin and Hassin (1994), which is concerned with finding the minimum-cost tour that visits a certain neighborhood of each target point. While certain special cases of the problem admit polynomial-time approximation schemes (see, e.g., Dumitrescu and Mitchell (2003), Mitchell (2007), Bodlaender et al. (2009), Chan and Jiang (2016)), the general version of the TSPN is NP-hard to approximate below a certain accuracy (see, e.g., de Berg et al. (2005), Safra and Schwartz (2006)). The reader is also referred to Gulczynski et al. (2006), Coutinho et al. (2016) for the related problem of close enough TSP.

Similar to Garone et al. (2010b, 2011) and Gambella et al. (2018), we focus on the variant of the CVTSP in which the order of target points is not a priori given and the Vehicle can visit one target point between each take-off and landing. The RBA of Gambella et al. (2018) can solve instances of the CVTSP with up to 15 target points to optimality, a relatively small number given the large scale of operations. For instance, following the earthquake and tsunami of 26 December 2004 in the Indian Ocean, 69 inhabited islands were impacted in the Maldives alone (see, e.g., UNEP (2005)). In this paper, we aim to develop exact and approximate solution approaches for larger instances of the CVTSP by identifying and exploiting specific structural properties of the problem. Our contributions are as follows. We establish several structural properties of the CVTSP. By exploiting these properties, we develop a new MISOCP optimization model. In contrast with the optimization model of Gambella et al. (2018) that employs assignment variables for determining the visiting order of target points, our formulation relies on routing variables and subtour elimination

constraints. We derive several optimality cuts arising from geometric observations. We also devise heuristic methods for computing near-optimal solutions for larger instances.

The rest of the paper is organized as follows. In Section 2, we give a formal definition of the CVTSP and identify several structural properties. We present improved and new formulations for the CVTSP in Section 3. Heuristic algorithms are presented in Section 4. Our computational experiments are reported in Section 5. Finally, we conclude the paper in Section 6.

2. Problem Definition and Structural Properties

In this section, we first give a formal definition of the CVTSP. Then, we present several structural properties that will later be utilized in our optimization formulation.

2.1. Problem Definition

As stated in the introduction, we consider a multi-vehicle system consisting of a slow Carrier with a long-range operational capability and a fast Vehicle with a limited range. The Carrier is able to transport, deploy, recover, and service the Vehicle. At the beginning of the mission, the Vehicle is assumed to be based on the Carrier located at an origin with full operational capability. Both the Carrier and the Vehicle should return to a prespecified destination at the end of the mission. The destination may, in general, be different from the origin.

The locations are specified by the (x, y) -coordinates on the plane. The Carrier can follow any continuous trajectory without exceeding its maximum speed. The Vehicle is transported by the Carrier whenever it is not deployed. Once the Vehicle is deployed, it can follow any continuous trajectory without exceeding its maximum speed and its maximum operating time. The Vehicle should return to the Carrier before exhausting its operational capability, which we assume to be instantaneously restored as soon as the Vehicle lands on the Carrier. During the mission, the Vehicle should visit a set of target points whose locations are assumed to be known. For each target point, the Vehicle takes off from the Carrier, travels to the location of the target point, and returns to the Carrier. We assume that the service time of the Vehicle at a target point is negligible. The objective is to plan the routes of the Carrier and the Vehicle in such a way that the mission is completed as quickly as possible.

The parameters of the problem are given in Table 1.

In Figure 1, we present an illustration of an optimal solution for LD21_3, one of the instances of the CVTSP with 20 target points used in our computational experiments in Section 5, where take-off, target, and landing points are denoted by empty, lightly-shaded, and fully-shaded vertices, respectively. For this instance, the origin and the destination, denoted by o and f , are at the same coordinates. The parameters regarding the Carrier's speed, the Vehicle's speed, and the Vehicle's

n	Number of target points
T	Set of target points ($T = \{1, \dots, n\}$)
q_j	Coordinates of the target point j , $j = 1, \dots, n$
V_v	Maximum speed of the Vehicle
V_c	Maximum speed of the Carrier ($V_c \leq V_v$)
a	Maximum operating time (autonomy) of the Vehicle
p_o	Coordinates of the origin
p_f	Coordinates of the destination

Table 1 Parameters of the CVTSP

autonomy in this example are $V_c = 1$, $V_v = 5$, $a = 1$. In Figure 1, the path of the Carrier is depicted by solid blue line segments whereas the path of the Vehicle is illustrated by dashed blue line segments. The optimal solution of the TSP on target points and the origin is denoted by red dotted line segments. As illustrated by this example, the optimal visit sequence of the target points in the CVTSP can be different from that of the optimal TSP tour due to the additional flexibility provided by the Vehicle. Therefore, solving the CVP using an optimal TSP tour on the target points may result in a suboptimal solution for the CVTSP. This example provides a justification for studying exact methods for the CVTSP.

2.2. Structural Properties

In this section, we prove several structural properties of the CVTSP, which we will utilize to develop a stronger formulation.

Let us denote by $p_{to,k}$ and $p_{l,k}$ the coordinates of the take-off and landing points of the Vehicle before and after visiting the target point in the k^{th} order in an optimal solution of the CVTSP, respectively, where $k = 1, \dots, n$. Therefore, the trajectory of the Carrier is given by

$$p_o, p_{to,1}, p_{l,1}, p_{to,2}, p_{l,2}, \dots, p_{to,n}, p_{l,n}, p_f.$$

As already observed in Garone et al. (2014), during any period in which the Vehicle is transported by the Carrier, the Carrier should clearly travel along straight line segments at its maximum speed denoted by V_c . It follows that the total time during which the Carrier transports the Vehicle is given by

$$t_{CV}^* = \frac{1}{V_c} \left(\|p_o - p_{to,1}\| + \sum_{k=1}^{n-1} \|p_{l,k} - p_{to,k+1}\| + \|p_{l,n} - p_f\| \right), \quad (1)$$

where $\|\cdot\|$ denotes the Euclidean norm.

Let us now focus on the total time required to visit a target point $j \in T$. Let us define $s_{to,j}$ and $s_{l,j}$ as in Table 2, which will be used frequently in the remainder of this section.

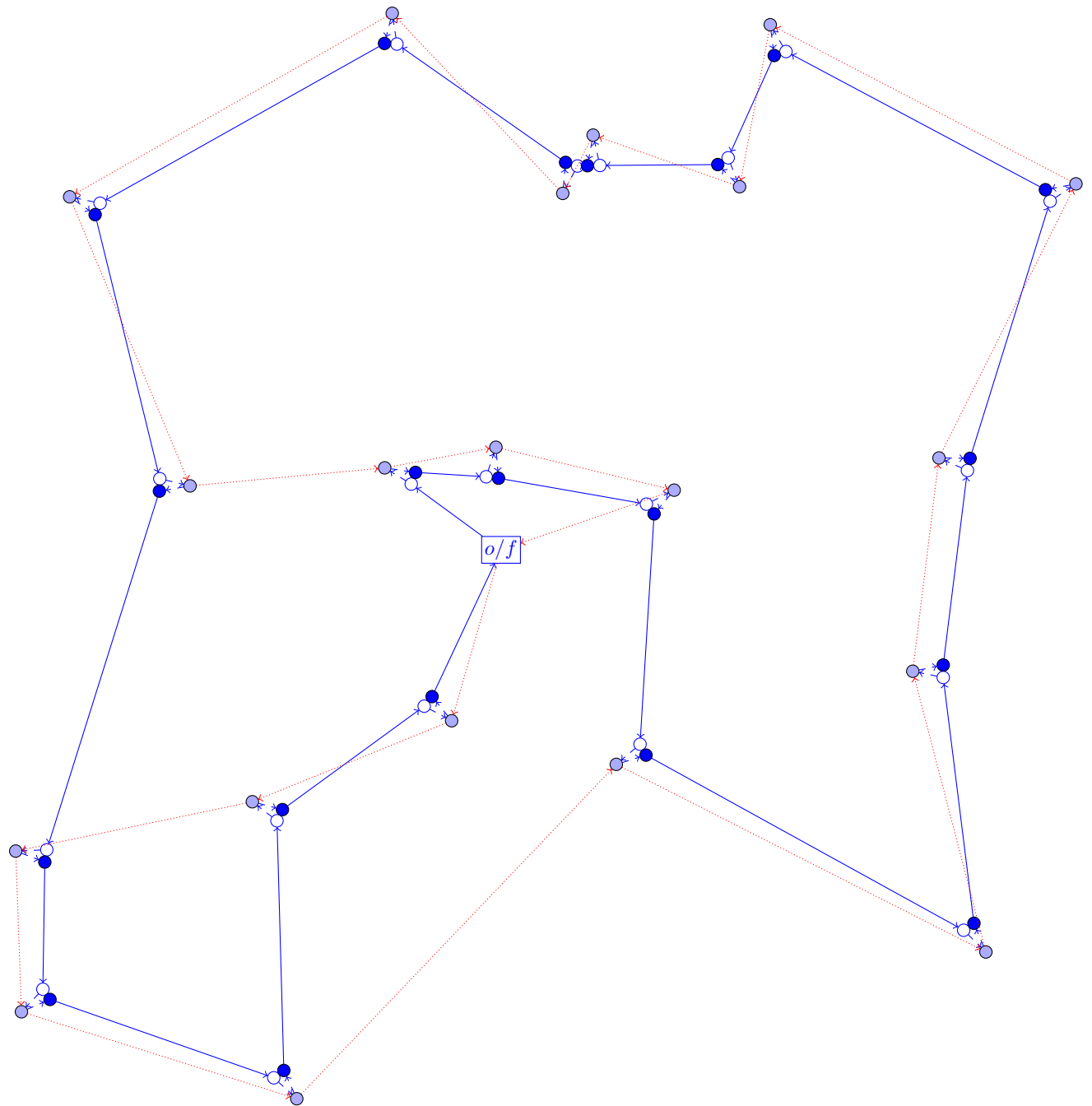


Figure 1 Optimal solutions of CVTSP (in blue) and TSP (in red) for the instance LD21_3

If the target point j is visited in the k^{th} order, then we have $s_{to,j} = p_{to,k}$ and $s_{l,j} = p_{l,k}$, where $j \in T$ and $k = 1, \dots, n$. The time elapsed during the visit of the target point j is given by

$$t_j^* = \max \{t_j^c, t_j^v\}, \quad j \in T, \quad (2)$$

$s_{to,j}$	Coordinates of the take-off point of the Vehicle right before visiting the target point j in an optimal solution of the CVTSP, $j \in T$
$s_{l,j}$	Coordinates of the landing point of the Vehicle right after visiting the target point j in an optimal solution of the CVTSP, $j \in T$

Table 2 **Definitions of $s_{to,j}$ and $s_{l,j}$, $j \in T$**

where t_j^c and t_j^v denote the minimum travel time of the Carrier and the Vehicle, respectively, and are given by

$$t_j^c = \frac{\|s_{to,j} - s_{l,j}\|}{V_c}, \quad t_j^v = \frac{\|s_{to,j} - q_j\| + \|q_j - s_{l,j}\|}{V_v}, \quad j \in T. \quad (3)$$

Due to the limited operational capability of the Vehicle, we have $t_j^* \leq a$, $j \in T$. We henceforth refer to t_j^* as the visit duration of target point j .

The optimal value of the CVTSP, which corresponds to the minimum completion time of the mission, is therefore given by

$$t^* = t_{CV}^* + \sum_{j \in T} t_j^*, \quad (4)$$

where t_{CV}^* and t_j^* are given by (1) and (2), respectively.

The following definition will be useful.

DEFINITION 1. Given an instance of the CVTSP, suppose that t_j^c and t_j^v are defined as in (3), where $j \in T$. Then,

- (i) if $t_j^c < t_j^v$, then the Carrier is said to wait for the Vehicle at target point j ,
- (ii) if $t_j^v < t_j^c$, then the Vehicle is said to wait for the Carrier at target point j ,
- (iii) if $t_j^c = t_j^v$, then the Carrier and the Vehicle are said to be perfectly synchronized at target point j .

The next result illustrates a useful property of an optimal solution of the CVTSP.

LEMMA 1. *In any optimal solution of the CVTSP, the Carrier never waits for the Vehicle at any target point, i.e., $t_j^v \leq t_j^c$ for each $j \in T$, where t_j^c and t_j^v are defined as in (3).*

Proof. Suppose, for a contradiction, that there exists an optimal solution of the CVTSP such that the Carrier waits for the Vehicle at some target point $j \in T$, i.e., $t_j^c < t_j^v = t_j^*$ by (2). Let us denote the take-off point before visiting target point j by to and the landing point after visiting target point j by l . Let us denote by i the previous point on the trajectory of the Carrier, which is either the landing point of the target point visited right before target point j or the origin. Figure 2 illustrates a schematic representation, where the solid lines represent the trajectory of the Carrier whereas the dashed lines correspond to that of the Vehicle. Note that both the take-off point and the landing point should be different from the target point j since we would otherwise have $t_j^v \leq t_j^c$ since $V_c \leq V_v$, contradicting our hypothesis.

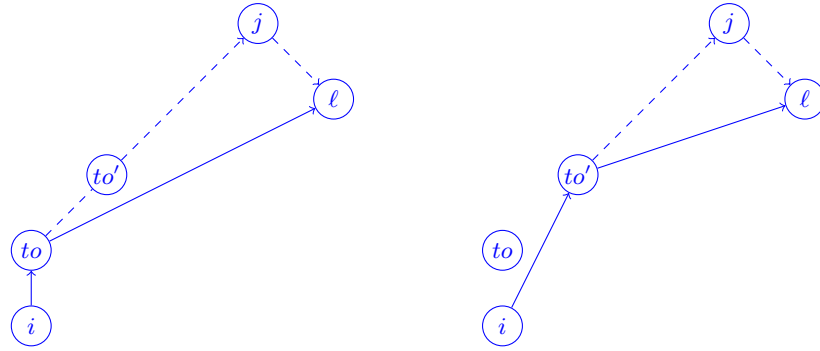


Figure 2 Illustration of the proof of Lemma 1

We make the following claim. There exists a point to' on the line segment between to and j such that deploying the Vehicle at to' as opposed to to strictly improves the objective function value. The coordinates of any point on the line segment between to and l , denoted by $to(\alpha)$, are given by $(1 - \alpha)s_{to,j} + \alpha q_j = s_{to,j} + \alpha(q_j - s_{to,j})$ for some $\alpha \in [0, 1]$, where q_j , $s_{to,j}$, and $s_{l,j}$ are defined as in Tables 1 and 2. Let us denote the travel time of the Carrier from to to $to(\alpha)$ and from $to(\alpha)$ to l by $t_c(\alpha)$ and the travel time of the Vehicle from $to(\alpha)$ to j and from j to l by $t_v(\alpha)$, where $\alpha \in [0, 1]$. Clearly, each of $t_c(\alpha)$ and $t_v(\alpha)$ is a continuous function of α and $t_v(\alpha)$ is a strictly decreasing function. Furthermore, $t_c(0) = t_j^c < t_v(0) = t_j^v$ and $t_c(1) \geq t_v(1)$ since $V_c \leq V_v$ and $s_{l,j} \neq q_j$. It follows that there exists $\alpha^* \in (0, 1]$ such that $t_c(\alpha^*) = t_v(\alpha^*) < t_v(0) = t_j^v \leq a$ since $t_v(\alpha)$ is strictly decreasing. Denoting $to(\alpha^*)$ by to' , if the Vehicle is deployed at to' instead of to , then the visit duration of target point j would be strictly smaller than t_j^v . Furthermore, note that using a shortcut from i to to' in the route of the Carrier will not increase the travel time of the Carrier by the triangle inequality. This contradicts the optimality of the original solution. $\square$

Our next result establishes the existence of an optimal solution of the CVTSP such that the Carrier and the Vehicle are perfectly synchronized at each target point.

PROPOSITION 1. *Given an instance of the CVTSP, there exists an optimal solution such that the Carrier and the Vehicle are perfectly synchronized at each target point, i.e., $t_j^v = t_j^c$ for each $j \in T$, where t_j^c and t_j^v are defined as in (3).*

Proof. Suppose that there exists an optimal solution such that the Carrier and the Vehicle are not perfectly synchronized at some target point $j \in T$. By Lemma 1, we have $t_j^v < t_j^c = t_j^*$, i.e., the Vehicle waits for the Carrier at target point j . Similar to the proof of Lemma 1, let us denote the take-off point before visiting target point j by to and the landing point after visiting target point j by l . Let us denote by i the previous point on the trajectory of the Carrier, which is either the landing point of the target point visited right before target point j or the origin. In Figure 3, the solid lines represent the trajectory of the Carrier whereas the dashed lines correspond to that of

the Vehicle. Note that at least one of the take-off and landing points should be different from the target point j since we would otherwise have $t_j^v = t_j^c = t_j^* = 0$, contradicting our hypothesis.

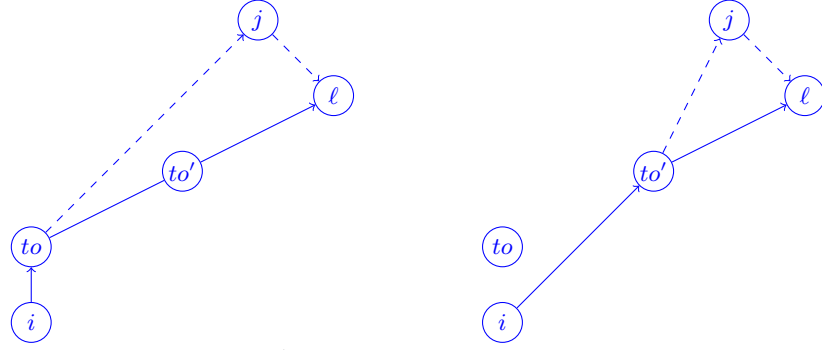


Figure 3 Illustration of the proof of Proposition 1

Following a similar argument as in the proof of Lemma 1, there exists a point to' on the line segment between to and ℓ such that the travel time of the Carrier and that of the Vehicle are identical and smaller than t_j^c if the Vehicle is deployed at to' instead of to . Finally, by the triangle inequality, we can use a short cut between i and to' in the trajectory of the Carrier without increasing its total travel time from i to ℓ . It follows that any optimal solution in which the Carrier and the Vehicle are not perfectly synchronized at a target point can be modified without worsening the objective function value and ensuring perfect synchronization. $\square$

Note that there may be an optimal solution of the CVTSP in which perfect synchronization may not necessarily be satisfied at a particular target point. Indeed, a close examination of the proof of Proposition 1 reveals that the objective function value remains the same if, for instance, the points i , to , and to' are collinear. Nevertheless, Proposition 1 ensures the existence of an optimal solution with perfect synchronization at *all* target points.

The next result is an immediate consequence of Proposition 1.

COROLLARY 1. *Given an instance of the CVTSP, there exists an optimal solution such that*

- (i) *the Carrier moves along straight line segments at its maximum speed V_c ,*
- (ii) *the Vehicle moves along straight line segments at its maximum speed V_v , and*
- (iii) *the Carrier and the Vehicle are perfectly synchronized at each target point.*

Next, we focus on the distance from the take-off or landing point to a target point. Denoting the visit duration of target point j by t_j^* in an optimal solution (see (2)), a naive upper bound on each of the two distances is given by $t_j^* V_v$. The next result provides a tighter upper bound.

PROPOSITION 2. *Given an instance of the CVTSP, any optimal solution satisfies*

$$\max \{ \|s_{to,j} - q_j\|, \|s_{l,j} - q_j\| \} \leq \frac{t_j^* (V_v + V_c)}{2} \leq \frac{a (V_v + V_c)}{2}, \quad j \in T, \quad (5)$$

where $s_{to,j}$ and $s_{l,j}$ are defined as in Table 2 and t_j^* is given by (2).

Proof. The second inequality in (5) directly follows from the inequality $t_j^* \leq a$, $j \in T$. For the first inequality, let us fix $j \in T$. By the triangle inequality,

$$\|s_{to,j} - q_j\| \leq \|s_{to,j} - s_{l,j}\| + \|s_{l,j} - q_j\| \quad (6)$$

$$= t_j^c V_c + t_j^v V_v - \|s_{to,j} - q_j\| \quad (7)$$

$$\leq t_j^* V_c + t_j^* V_v - \|s_{to,j} - q_j\|, \quad (8)$$

where we used (3) in the second line and (2) in the last line. It follows that $\|s_{to,j} - q_j\| \leq t_j^* (V_v + V_c) / 2$. The upper bound on $\|s_{l,j} - q_j\|$ can be obtained in a similar manner. $\square$

The next corollary establishes the existence of an optimal solution in which the distance from the take-off or landing point to each target point can be bounded below.

COROLLARY 2. *Given an instance of the CVTSP, there exists an optimal solution such that*

$$\min \{ \|s_{to,j} - q_j\|, \|s_{l,j} - q_j\| \} \geq \frac{t_j^* (V_v - V_c)}{2}, \quad j \in T, \quad (9)$$

where $s_{to,j}$ and $s_{l,j}$ are defined as in Table 2 and t_j^* is given by (2).

Proof. By Corollary 1, there exists an optimal solution of the CVTSP such that the Carrier and the Vehicle are perfectly synchronized at each target point. For such an optimal solution,

$$\|s_{to,j} - q_j\| + \|s_{l,j} - q_j\| = V_v t_j^*, \quad j \in T.$$

The relation (9) follows immediately from (5) and the equation above. $\square$

We next introduce the following definition.

DEFINITION 2. Given an instance of the CVTSP, the Vehicle is said to use its full autonomy at target point j in an optimal solution if $t_j^v = a$, where $j \in T$, and t_j^v is given by (3).

The next corollary follows from Proposition 2.

COROLLARY 3. *Given an instance of the CVTSP, suppose that $s_{to,j}$ and $s_{l,j}$ are defined as in Table 2, where $j \in T$. If*

$$\max \{ \|s_{to,j} - q_j\|, \|s_{l,j} - q_j\| \} = \frac{a (V_v + V_c)}{2}, \quad (10)$$

then the Vehicle uses its full autonomy at target point j and the Carrier and the Vehicle are perfectly synchronized at target point j .

Proof. By Proposition 2, we obtain $t_j^* = a$. Since $t_j^* = \max\{t_j^v, t_j^c\} = a$ and $t_j^v \leq t_j^c$ by Lemma 1, we have $t_j^v \leq t_j^c = a$. If $t_j^v < t_j^c$, then, by (6)–(8), we would obtain

$$\|s_{to,j} - q_j\| < aV_c + aV_v - \|s_{to,j} - q_j\|,$$

which would imply that $\|s_{to,j} - q_j\| < a(V_v + V_c)/2$. A similar argument would yield $\|s_{l,j} - q_j\| < a(V_v + V_c)/2$, which would contradict (10). It follows that $t_j^v = t_j^c = a$, i.e., the Carrier and the Vehicle are perfectly synchronized at target point j . $\square$

The next technical lemma establishes a geometric property that will be useful in the proof of the subsequent proposition.

LEMMA 2. *Given an instance of the CVTSP, suppose that the Carrier and the Vehicle are perfectly synchronized at a target point $j \in T$. Consider the triangle with the base given by the line segment between $s_{to,j}$ and $s_{l,j}$ and the apex given by q_j , where $s_{to,j}$ and $s_{l,j}$ are defined as in Table 2 and q_j is defined as in Table 1. The height of this triangle, denoted by h_j , satisfies*

$$h_j \leq \frac{t_j^*}{2} \sqrt{V_v^2 - V_c^2}, \quad (11)$$

where t_j^* is given by (2).

Proof. Suppose that the Carrier and the Vehicle are perfectly synchronized at a target point $j \in T$. If $t_j^* = 0$, then $s_{to,j} = s_{l,j} = q_j$, which implies that the triangle degenerates to a point and $h_j = 0$, clearly satisfying (11).

Suppose now that $t_j^* > 0$. Consider the triangle with the base given by the line segment between $s_{to,j}$ and $s_{l,j}$ and the apex given by q_j . Let us denote the length of the base by α and the lengths of the other two sides by β and γ . By perfect synchronization,

$$\begin{aligned} \alpha &= V_c t_j^*, \\ \beta + \gamma &= V_v t_j^*. \end{aligned}$$

The area of this triangle is given by $(\alpha h_j)/2$. By Heron's formula,

$$\frac{\alpha h_j}{2} = \sqrt{s(s-\alpha)(s-\beta)(s-\gamma)},$$

where $s = (\alpha + \beta + \gamma)/2 = ((V_v + V_c)t_j^*)/2$. Therefore,

$$h_j^2 = \frac{4s(s-\alpha)(s-\beta)(s-\gamma)}{\alpha^2}$$

$$\begin{aligned}
 &= \frac{4[((V_v + V_c)t_j^*)/2][((V_v - V_c)t_j^*)/2](s - \beta)(s - \gamma)}{V_c^2(t_j^*)^2} \\
 &= \frac{(V_v^2 - V_c^2)(s - \beta)(s - \gamma)}{V_c^2}
 \end{aligned}$$

By the triangle inequality, $s - \beta \geq 0$ and $s - \gamma \geq 0$. The last term is clearly maximized if $\beta = \gamma = (V_v t_j^*)/2$, which implies that

$$h_j^2 \leq \frac{(V_v^2 - V_c^2)[(V_c t_j^*)/2]^2}{V_c^2} = \frac{(V_v^2 - V_c^2)(t_j^*)^2}{4},$$

which establishes (11). $\square$

The next result allows us to identify a subset of the target points at which the Vehicle uses its full autonomy in an optimal solution.

PROPOSITION 3. *Given an instance of the CVTSP, if a target point j , where $j \in T$, satisfies all of the following conditions*

- (i) $\|p_o - q_j\| \geq \frac{a(V_v + V_c)}{2}$,
- (ii) $\|q_i - q_j\| \geq a(V_v + V_c), \forall i \in T \setminus \{j\}$, and
- (iii) $\|p_f - q_j\| \geq \frac{a(V_v + V_c)}{2}$,

then there exists an optimal solution such that the Vehicle uses its full autonomy at target point j and the Carrier and the Vehicle are perfectly synchronized at target point j .

For the sake of brevity, we provide the proof of Proposition 3 in the Appendix. The next result generalizes the sufficient conditions for full autonomy given in Proposition 3 by taking into account the visit sequence of the target points.

PROPOSITION 4. *There exists an optimal solution of the CVTSP such that the Vehicle uses its full autonomy at target point j , where $j \in T$, if the Carrier travels*

- (i) *from the origin to target point j , and $\|p_o - q_j\| \geq \frac{a(V_v + V_c)}{2}$, or*
- (ii) *from target point $i \in T \setminus \{j\}$ to target point j , and $\|q_i - q_j\| \geq a(V_v + V_c)$,*
and
- (iii) *from target point j to target point $i \in T \setminus \{j\}$, and $\|q_i - q_j\| \geq a(V_v + V_c)$, or*
- (iv) *from target point j to the destination, and $\|q_j - p_f\| \geq \frac{a(V_v + V_c)}{2}$.*

The proof of Proposition 4 is also provided in the Appendix.

3. Formulations

As stated in the introduction, the formulation of Gambella et al. (2018) employs assignment variables for sequencing target points. In this section, we present a new optimization model for the CVTSP that relies on routing variables. In addition, we utilize the structural properties presented in Section 2 to present several optimality cuts.

3.1. A Mixed Integer Nonlinear Programming Formulation

In this section, we present a new optimization model for the CVTSP. We use the same parameters presented in Table 1. Our decision variables are presented in Table 3.

x_{ij}	Binary variable which is equal to 1 if target point i is visited right before target point j and 0 otherwise, $i \in T$; $j \in T$
$x_{o,j}$	Binary variable which is equal to 1 if target point j is visited right after the origin (i.e., j is the first target point to be visited) and 0 otherwise, $j \in T$
$x_{j,f}$	Binary variable which is equal to 1 if target point j is visited right before the destination (i.e., j is the last target point to be visited) and 0 otherwise, $j \in T$
$s_{to,j}$	Coordinates of the take-off point of the Vehicle before visiting the target point j , $j \in T$
$s_{l,j}$	Coordinates of the landing point of the Vehicle after visiting the target point j , $j \in T$
ρ_{ij}	Distance traveled by the Carrier from target point i to target point j if target point j is visited right after target point i , $i \in T$; $j \in T$
$\rho_{o,j}$	Distance traveled by the Carrier from the origin to target point j if target point j is visited right after the origin, $j \in T$
$\rho_{j,f}$	Distance traveled by the Carrier from target point j to the destination if target point j is visited right before the destination, $j \in T$
t_j	Visit duration of target point j , $j \in T$

Table 3 Decision variables of CVTSP1

Next, we present our alternative formulation, denoted by CVTSP1:

$$\min \sum_{j \in T} t_j + \sum_{i \in T} \sum_{j \in T} (1/V_c) \rho_{ij} + \sum_{j \in T} (1/V_c) (\rho_{o,j} + \rho_{j,f}) \quad (12)$$

s.t.

$$\|s_{to,j} - s_{l,j}\| \leq V_c t_j, \quad j \in T \quad (13)$$

$$\|s_{to,j} - q_j\| + \|s_{l,j} - q_j\| \leq V_v t_j, \quad j \in T \quad (14)$$

$$\|s_{to,j} - q_j\| \leq (1/2) (V_v + V_c) t_j, \quad j \in T \quad (15)$$

$$\|s_{l,j} - q_j\| \leq (1/2) (V_v + V_c) t_j, \quad j \in T \quad (16)$$

$$\|p_o - s_{to,j}\| \leq \rho_{o,j} + (1 - x_{o,j}) (\|p_o - q_j\| + (1/2) (V_v + V_c) a), \quad j \in T \quad (17)$$

$$x_{o,j} \|p_o - q_j\| \leq \rho_{o,j} + (1/2) (V_v + V_c) t_j, \quad j \in T \quad (18)$$

$$\|s_{l,j} - p_f\| \leq \rho_{j,f} + (1 - x_{j,f}) (\|q_j - p_f\| + (1/2) (V_v + V_c) a), \quad j \in T \quad (19)$$

$$x_{j,f} \|q_j - p_f\| \leq \rho_{j,f} + (1/2) (V_v + V_c) t_j, \quad j \in T \quad (20)$$

$$\|s_{to,j} - s_{l,i}\| \leq \rho_{ij} + (1 - x_{ij}) (\|q_j - q_i\| + (V_v + V_c) a), \quad i \in T, j \in T \quad (21)$$

$$x_{ij} \|q_j - q_i\| \leq \rho_{ij} + (1/2) (V_v + V_c) (t_i + t_j), \quad i \in T, j \in T \quad (22)$$

$$x_{jj} = 0, \quad j \in T \quad (23)$$

$$x_{o,j} + \sum_{i \in T} x_{ij} = 1, \quad j \in T \quad (24)$$

$$x_{j,f} + \sum_{k \in T} x_{jk} = 1, \quad i \in T \quad (25)$$

$$\sum_{i \in S} \sum_{j \in S} x_{ij} \leq |S| - 1, \quad S \subseteq T; \quad 2 \leq |S| \leq n \quad (26)$$

$$t_j \leq a, \quad j \in T \quad (27)$$

$$\rho_{ij} \geq 0, \quad i \in T, j \in T \quad (28)$$

$$\rho_{o,j} \geq 0, \quad j \in T \quad (29)$$

$$\rho_{j,f} \geq 0, \quad j \in T \quad (30)$$

$$t_j \geq 0, \quad j \in T \quad (31)$$

$$x_{ij} \in \{0, 1\}, \quad i \in T, j \in T \quad (32)$$

$$x_{o,j} \in \{0, 1\}, \quad j \in T \quad (33)$$

$$x_{j,f} \in \{0, 1\}, \quad j \in T. \quad (34)$$

The objective function (12) is composed of the sum of the total travel time of the Carrier along its trajectory while transporting the Vehicle and the total visit duration of the target points. The constraints (13) and (14) correspond to the visit duration of the target points. The constraints (15) and (16) directly follow from Proposition 2. If the target point j is the first target point to be visited (i.e., if $x_{o,j} = 1$), then $\rho_{o,j}$ corresponds to the distance traveled by the Carrier from the origin to the target point j by (17). Otherwise, $\rho_{o,j}$ can be set to zero by the triangle inequality and Proposition 2. The constraint (18) is redundant if $x_{o,j} = 1$. Otherwise, it reduces to the triangle inequality between the origin, take-off point before visiting target point j , and target point j by Proposition 2, thereby providing a lower bound on $\rho_{o,j}$. The constraints (19) and (20) account for the last target point to be visited and are structured similarly to (17) and (18), respectively. The distance traveled by the Carrier between the landing point of target point i and the take-off point of target point j is reflected in the constraints (21) and (22) in a similar manner. The constraints (23)–(26) ensure that all target points are visited in a proper order starting from the origin and ending at the destination. Note that we employ subtour elimination constraints (26) since our model relies on routing variables x_{ij} . The maximum operating time of the Vehicle is reflected in (27). Finally, the constraints (28)–(34) specify the ranges of the decision variables.

CVTSP1 is a mixed integer second order conic optimization model consisting of $n^2 + 5n$ linear constraints, $2^n - n - 1$ subtour elimination constraints, $n^2 + 7n$ three-dimensional second order

conic constraints, $n^2 + 3n$ nonnegative continuous variables, $4n$ general continuous variables, and $n^2 + 2n$ binary variables.

3.2. Optimality Cuts

In this section, we present a set of optimality cuts for CVTSP1. Let us define the following index sets:

$$\mathcal{I}_j = \{i \in \{1, \dots, n\} : \|q_i - q_j\| \geq a(V_v + V_c)\}, \quad j \in T, \quad (35)$$

$$\mathcal{I}_o = \{i \in \{1, \dots, n\} : \|p_o - q_i\| \geq a(V_v + V_c)/2\}, \quad (36)$$

$$\mathcal{I}_f = \{i \in \{1, \dots, n\} : \|p_f - q_i\| \geq a(V_v + V_c)/2\}. \quad (37)$$

PROPOSITION 5. *Given an instance of the CVTSP, there exists an optimal solution of CVTSP1 that satisfies each of the following inequalities:*

$$t_j \geq a \left(x_{o,j} + \sum_{i \in \mathcal{I}_j} (x_{ij} + x_{ji}) + x_{j,f} - 1 \right), \quad \forall j \in \mathcal{I}_o \cap \mathcal{I}_f, \quad (38)$$

$$t_j \geq a \left(\sum_{i \in \mathcal{I}_j} (x_{ij} + x_{ji}) + x_{j,f} - 1 \right), \quad \forall j \in (T \setminus \mathcal{I}_o) \cap \mathcal{I}_f, \quad (39)$$

$$t_j \geq a \left(x_{o,j} + \sum_{i \in \mathcal{I}_j} (x_{ij} + x_{ji}) - 1 \right), \quad \forall j \in \mathcal{I}_o \cap (T \setminus \mathcal{I}_f), \quad (40)$$

$$t_j \geq a \left(\sum_{i \in \mathcal{I}_j} (x_{ij} + x_{ji}) - 1 \right), \quad \forall j \in (T \setminus \mathcal{I}_o) \cap (T \setminus \mathcal{I}_f). \quad (41)$$

Proof. The proof directly follows from Proposition 4 and the constraints (24) and (25). $\square$

For the next proposition, we introduce the auxiliary variables presented in Table 4.

$\delta_{to,j}$	Distance traveled by the Vehicle from the take-off point $s_{to,j}$ to target point j , $j \in T$
$\delta_{l,j}$	Distance traveled by the Vehicle from target point j to the landing point $s_{l,j}$ to , $j \in T$

Table 4 Auxiliary decision variables of CVTSP1

PROPOSITION 6. *Given an instance of the CVTSP, there exists an optimal solution of CVTSP1 that satisfies each of the following relations:*

$$\|s_{to,j} - q_j\| \leq \delta_{to,j}, \quad j \in T, \quad (42)$$

$$\|s_{l,j} - q_j\| \leq \delta_{l,j}, \quad j \in T, \quad (43)$$

$$\delta_{to,j} + \delta_{l,j} = V_v t_j, \quad j \in T, \quad (44)$$

$$\delta_{to,j} \geq (1/2)(V_v - V_c)t_j, \quad j \in T, \quad (45)$$

$$\delta_{l,j} \geq (1/2)(V_v - V_c)t_j, \quad j \in T. \quad (46)$$

Proof. The inequalities (42) and (43) follow from the definitions of $\delta_{to,j}$ and $\delta_{l,j}$ in Table 4. The equality (44) follows from Corollary 1 and the inequalities (45) and (46) are implied by Corollary 2.

□

Note that we refer to the relations in Propositions 5 and 6 as optimality cuts since there may exist feasible (even optimal) solutions of CVTSP1 that violate these relations. For instance, consider a CVTSP instance with one target point, where $p_o = (0,0)$, $p_f = (4,0)$, $q_1 = (2,0)$, $V_c = 1$, $V_v = 2$, and $a = 1$. Clearly, in any optimal solution of the CVTSP, the Carrier moves along a straight line segment from the origin to the destination through the target point. As such, the Vehicle does not need to leave the Carrier. However, by Propositions 5 and 6, we can impose $t_1 = a = 1$, which implies that there are only two optimal solutions that satisfy the given optimality cuts. Either the Vehicle takes off at $(0.5,0)$, visits the target point, and lands on the Carrier at $(1.5,0)$, or the Vehicle takes off at $(0,2.5)$, visits the target point, and lands on the Carrier at $(3.5,0)$. Note that, in both optimal solutions, the Vehicle uses full autonomy, moves along straight line segments, and is perfectly synchronized with the Carrier. As illustrated by this simple example, these relations may reduce the feasible region and our results in Section 2 imply the existence of an optimal solution that satisfies each of these relations.

We have also attempted to utilize the lower bound presented by Garone et al. (2010a) and used by Gambella et al. (2018) for the case when the order of the target points is fixed. Denoting the order ord , and the total length of the TSP tour corresponding to ord as $TSP(ord)$, the lower bound is:

$$LB(ord) = \frac{TSP(ord)}{V_c} - n \frac{V_v a}{V_c} + na. \quad (47)$$

In order to generalize this lower bound, we have replaced $TSP(ord)$ with the TSP distance of the target points based on the routing variables. The resulting optimality cut is

$$\begin{aligned} \sum_{j \in T} t_j + \sum_{i \in T} \sum_{j \in T} (1/V_c) \rho_{ij} + \sum_{j \in T} (1/V_c) (\rho_{0,j} + \rho_{j,f}) \geq \\ \frac{\sum_{i \in T} \|p_o - q_i\| x_{o,i} + \sum_{i \in T} \sum_{j \in T} \|q_i - q_j\| x_{i,j} + \sum_{j \in T} \|q_j - p_f\| x_{j,f}}{V_c} - n \frac{V_v a}{V_c} + na. \end{aligned} \quad (48)$$

However, this optimality cut was not observed to improve the lower bound or the solution time of our models except for a subset of instances.

4. Heuristic Algorithms

In this section, we provide a constructive algorithm to generate an initial solution, and an Iterated Local Search (ILS) algorithm to find high quality solutions for larger instances of the CVTSP.

4.1. Constructive Algorithm

We start by a short discussion of the CVP, which can be solved in polynomial time to find the optimal take-off and landing points for a given sequence of target points. Hence, it is theoretically possible to only search over the possible sequences of target points and use the CVP to determine the take-off and landing points for each sequence. However, being a second order conic optimization problem, the CVP still requires a non-negligible amount of CPU time. Table 5, which presents the results of our computational experiments with the CVP with varying number of target points, clearly reveals that the average CPU time requirement makes it impractical to repeatedly solve the CVP within a local search operator.

Table 5 Average CPU time requirement to solve a CVP instance for $|T|$

$ T $	CPU time (seconds)
10	0.07
20	0.56
30	1.5
40	4.92
50	12.62
60	32.65
70	76.06
80	153.75
90	318.36
100	645.82

On the other hand, solving the CVP for a limited number of times can significantly improve the quality of solutions of a heuristic algorithm. An intuitive constructive heuristic for the CVTSP is to solve the Hamiltonian Path Problem (HPP) from the origin to the destination through the target points, and solve the CVP for the resulting sequence. We remark that a similar constructive heuristic algorithm was also employed in Garone et al. (2010b).

We denote by σ the sequence of target points to be visited in a solution, where $\sigma(i)$ corresponds to the index of the target point to be visited in the i^{th} order. Finally, for the sake of brevity, we will refer to the vector of all landing and take-off coordinates as $\mathbf{s}$. This constructive heuristic is presented in Algorithm 1.

Algorithm 1 Constructive heuristic

- 1: Solve a Hamiltonian Path Problem from o to f through T to determine σ^*
 - 2: Solve an instance of the CVP for σ^* to find $\mathbf{s}^*$
-

We remark that an optimal solution of the HPP does not necessarily yield a high quality solution of the CVTSP by itself, and can deviate significantly from the optimal sequence for the CVTSP solution, as shown in Figure 4. We have also observed that the optimal objective function value of the HPP deviates by 11.76% from the optimal value of the CVTSP on average for the instances with known optimal solutions. Therefore, we propose an iterated local search algorithm in the next section in an attempt to improve the solution constructed by Algorithm 1.

4.2. Iterated Local Search algorithm

Iterated Local Search (ILS) is a metaheuristic algorithm that has been successfully used for solving many variants of the TSP and the Vehicle Routing Problem (VRP). It is based on the idea of perturbing a given solution and re-optimizing through local search operators. We refer the interested reader to the comprehensive paper by Subramanian et al. (2013). Most of the ILS implementations for the VRP use the sequence of the customers to represent solutions. We will write σ^* to denote the best known sequence, and $\mathbf{s}^*$ to denote the associated take-off and landing coordinates. Finally, we will denote the objective function value corresponding to a sequence σ and the associated take-off and landing coordinates $\mathbf{s}$ as $z(\sigma, \mathbf{s})$.

In our algorithmic design, we have opted to implement three well-known local search operators to improve the sequence, namely, 1-OPT (removing a target point in the sequence and reinserting it in a different position), 2-OPT (exchanging two carrier arcs in the route, which corresponds to reversing a section of the sequence), and 2-EXCHANGE (swapping two target points in the sequence) (Groër et al. 2010), and choose the best sequence found among them in each iteration. For 1-OPT and 2-EXCHANGE, we have implemented the operators to retain the take-off and landing coordinates of each target point, effectively preserving feasibility. In our implementation of 2-OPT, we have evaluated the resulting solutions by swapping the take-off and landing points of each target point within the reversed section. This approach has allowed us to attain better solutions while still preserving feasibility.

As pointed out in Section 4.1, for any given sequence σ , it is possible to determine $\mathbf{s}$ by iteratively solving instances of the CVP. However, repeatedly recomputing take-off and landing points using the CVP results in computational overhead, especially when implemented within a local search operator. To balance the trade-off between the computational effort and the search for better solutions, we adopt the following two approaches. First, we compute the take-off and landing points only at the beginning of each local search iteration. Second, rather than solving the CVP, we

construct a solution of the CVP by iteratively solving a basic variant of the CVP, which we refer to as CVP-B, to determine the take-off and landing points.

The CVP-B is a special case of the CVP with only one target point. Devoid of the need to sequence the target points, CVP-B can be formulated as a second order conic optimization model with four variables that correspond to the coordinates of the take-off and the landing points of the Vehicle, and four constraints given by (13), (14), (15), (16). We will refer to an instance of CVP-B starting from q_i , visiting q_j , and ending at q_k as $\text{CVP-B}(q_i, q_j, q_k)$. Recall that σ denotes the sequence of target points to be visited in a solution, where $\sigma(i)$ corresponds to the index of the target point to be visited in the i^{th} order. For a given sequence σ , we first solve $\text{CVP-B}(p_o, q_{\sigma(1)}, q_{\sigma(2)})$ to determine the take-off and landing points for $q_{\sigma(1)}$. Next, we treat the landing point of $q_{\sigma(1)}$ as the origin, $q_{\sigma(2)}$ as the target point, and $q_{\sigma(3)}$ as the destination in order to compute the take-off and landing points for $q_{\sigma(2)}$. We continue in an iterative manner until we solve the CVP-B to compute the take-off and landing points for $q_{\sigma(n)}$ by treating p_f as the final destination. Therefore, we construct a feasible solution of CVP by instead solving $|T| + 1$ instances of the simpler variant CVP-B.

In the ILS algorithm we present below, we exactly solve the CVP twice: Once at the beginning for the initial HPP sequence (Algorithm 1), and once at the end for the best known sequence to ensure the optimality of the take-off and landing points.

A crucial component of ILS is the perturbation step, where we have used the same operators to move to random solutions, and chose 1 to 3 such moves for each operator. We have set the number of iterations $k_{max} = 2|T|$, which we have observed to provide high quality solutions without requiring extensive CPU time. The details of the algorithm are presented as Algorithm 2.

5. Computational Results

In our computational experiments, we used CPLEX 12.8.0 as a mixed integer second order conic optimization solver. We have conducted experiments on the nodes of the computing cluster *Balena* hosted at the University of Bath, with Intel E5-2650 v2 CPUs at a speed of 2.60 GHz. We have implemented the model of Gambella et al. (2018) on the same platform, and limited both models to use a single thread, in order to ensure a fair comparison. In all tables below, the acronyms LB and UB correspond to lower bound and upper bound, respectively.

5.1. Models

We have started by testing the model of Gambella et al. (2018), CVTSP1, as well as CVTSP1' that includes the optimality cuts (38) - (46). All models were tested on the instance sets kindly provided by the authors of Gambella et al. (2018), and we imposed a CPU time limit of 3600 seconds.

Algorithm 2 Iterated Local Search

```

1: Invoke Algorithm 1 to determine  $\sigma^*$  and  $\mathbf{s}^*$ 
2: For  $k = 1$  to  $k_{max}$  do
3:    $\sigma = \sigma^*$ ;  $\mathbf{s} = \mathbf{s}^*$ 
4:   Perturb  $\sigma$  by applying random 1-OPT, 2-OPT, and 2-EXCHANGE moves.
5:   Do
6:     Solve CVP-B( $p_o, q_{\sigma(1)}, q_{\sigma(2)}$ ) to determine  $s_{to, \sigma(1)}$  and  $s_{l, \sigma(1)}$ 
7:     For  $ord = 2$  to  $|T| - 1$  do
8:       Solve CVP-B( $s_{l, \sigma(ord-1)}, q_{\sigma(ord)}, q_{\sigma(ord+1)}$ ) to determine  $s_{to, \sigma(ord)}$  and  $s_{l, \sigma(ord)}$ 
9:     End For
10:    Solve CVP-B( $s_{l, \sigma(n-1)}, q_{\sigma(n)}, p_f$ ) to determine  $s_{to, \sigma(n)}$  and  $s_{l, \sigma(n)}$ 
11:     $\sigma' = \sigma$ 
12:    Apply local search to  $\sigma$  using 1-OPT, 2-OPT, and 2-EXCHANGE operators.
13:    While  $\sigma \neq \sigma'$ 
14:      If  $z(\sigma, \mathbf{s}) < z(\sigma^*, \mathbf{s}^*)$  Then
15:         $\sigma^* = \sigma$ ;  $\mathbf{s}^* = \mathbf{s}$ 
16:      End If
17:    End For
18: Solve an instance of the CVP for  $\sigma^*$  to update  $\mathbf{s}^*$ 

```

The subtour elimination constraints (26) were implemented as *connectivity constraints* from the origin to each target point, and from each target point to the destination. Violated connectivity constraints were separated by solving a max-flow problem on the residual graph generated by the fractional x_{ij} values at each node of the branch-and-cut tree, similar to the implementation in Battarra et al. (2010). The relations (38) - (46) were directly added to the model for the target points that are *distant*, i.e. $j \in T : j \in \mathcal{I}_o, j \in \mathcal{I}_f, \mathcal{I}_j = T \setminus \{j\}$, and passed on to CPLEX as user cuts for the rest of the target points.

The instances from Gambella et al. (2018) are comprised of four families, denoted by SD, MD, LD, and VLD. In each family, instances are randomly generated in the following fashion. In SD and MD, target point coordinates are uniformly generated in $[-25, 25]$ and $[-25, 25]$, respectively. On the other hand, in LD and VLD, larger intervals of $[-50, 50]$ and $[-50, 50]$ are used to generate the target point coordinates. In the families MD and VLD, target points are chosen to satisfy $\|q_i - q_j\| \geq V_v a$ for each $i \in T$ and $j \in T$, where $i \neq j$, whereas no such restriction is imposed in the families SD and LD. Each instance is denoted by FAM_N_NUM, where $\text{FAM} \in \{\text{SD}, \text{MD}, \text{LD}, \text{VLD}\}$, $N = n + 1$ with $N \in \{11, 12, \dots, 16\}$, and $\text{NUM} \in \{1, 2, 3\}$. Therefore, a total of 72 instances

were tested. In each instance, the parameters were given by $V_c = 1, V_v = 5, a = 1, p_o = [0, 0]^T$, and $p_f = [0, 0]^T$.

Detailed results are presented in Tables 6, 7, 8, and 9, where the best CPU time (in seconds) for each instance is indicated in boldface. Notably, we did not improve upon any of the upper bounds found by Gambella et al. (2018), hence we denote the upper bounds under the heading ‘‘Gambella RBA UB’’. For instance sets LD and VLD, CVTSP1 improves the average CPU time requirement of Gambella et al. (2018) by factors of 8.75 and 9.30, respectively. In addition, CVTSP1 successfully solves a previously unsolved instance with $|T| = 15$. Furthermore, CVTSP1’ augments these results to improvement factors of 17.58 and 12.78, respectively. Notably, the model of Gambella et al. (2018) is consistently faster for smaller instances, and the factors of improvement are due to instances with $|T| \in \{14, 15\}$. More modest improvements are observed for the instance sets MD and SD, with similar performances in terms of instance size. In these instance sets, CVTSP1’ can successfully solve 2 previously unsolved instances. We conclude that CVTSP1’ clearly outperforms both the MISOC model of Gambella et al. (2018) and CVTSP1 for these instances.

Table 6 Comparison of the models for SD instances

Instance	Gambella RBA UB	Gambella MISOC CPU	CVTSP1 CPU	CVTSP1’ CPU
SD11.1	108.76	1.96	41.64	17.06
SD11.2	113.92	8.62	91.36	68.36
SD11.3	125.19	1.93	28.45	15.16
SD12.1	107.60	2.28	20.41	12.47
SD12.2	153.94	1.96	58.67	23.67
SD12.3	118.61	8.76	32.33	22.56
SD13.1	116.12	4.12	37.91	22.84
SD13.2	136.86	137.52	689.08	919.78
SD13.3	121.47	13.11	84.08	36.53
SD14.1	128.14	201.05	471.73	128.36
SD14.2	124.66	152.53	357.42	134.48
SD14.3	138.07	103.62	82.94	69.44
SD15.1	123.67	403.20	359.25	240.83
SD15.2	136.10	643.02	592.08	436.47
SD15.3	132.72	3600.00	3600.00	3600.03
SD16.1	145.10	3600.00	379.55	346.86
SD16.2	155.36	3600.00	3600.00	2228.28
SD16.3	128.38	3600.00	3600.00	2060.22
Average		893.54	784.83	576.86

We are not able to provide a direct comparison of our results with those of the RBA of Gambella et al. (2018), due to the difference of the computing platforms used. However, the CPU speed of the computer (3.10 GHz) they have used is faster than our CPU by a factor of 1.2. For the sake of brevity, we will forgo an instance-by-instance comparison and will compare only average results, as presented in Table 10. CVTSP1’ is observed to provide significant improvement factors for all

Table 7 Comparison of the models for MD instances

Instance	Gambella RBA UB	Gambella MISOC CPU	CVTSP1 CPU	CVTSP1' CPU
MD11_1	146.85	0.19	9.42	5.28
MD11_2	132.12	0.60	31.58	9.53
MD11_3	133.42	0.76	29.16	10.86
MD12_1	157.74	2.15	21.27	15.86
MD12_2	165.63	1.98	24.02	23.69
MD12_3	121.24	1.58	34.77	19.03
MD13_1	150.89	15.25	77.94	63.75
MD13_2	130.99	16.92	74.94	42.75
MD13_3	150.37	12.72	46.08	29.38
MD14_1	146.95	24.23	169.53	65.61
MD14_2	163.59	54.06	146.58	83.95
MD14_3	153.01	38.34	89.48	44.95
MD15_1	168.24	1476.65	2538.06	789.94
MD15_2	136.94	1277.98	493.59	234.95
MD15_3	157.86	624.48	1118.97	656.53
MD16_1	166.21	3600.00	3600.00	1716.09
MD16_2	177.23	3232.09	786.81	420.25
MD16_3	64.37	3600.00	475.14	198.94
Average		776.67	542.63	246.19

Table 8 Comparison of the models for LD instances

Instance	Gambella RBA UB	Gambella MISOC CPU	CVTSP1 CPU	CVTSP1' CPU
LD11_1	311.55	0.06	4.24	2.45
LD11_2	345.19	0.23	5.09	3.84
LD11_3	299.53	0.21	2.41	4.52
LD12_1	296.07	0.29	6.92	4.77
LD12_2	308.26	0.69	6.77	6.53
LD12_3	270.31	0.16	3.27	3.67
LD13_1	261.64	1.43	13.28	10.98
LD13_2	294.85	2.16	8.30	8.69
LD13_3	307.34	2.39	10.83	7.64
LD14_1	319.80	5.84	11.75	7.94
LD14_2	282.91	23.94	47.77	28.98
LD14_3	301.60	5.35	11.39	8.42
LD15_1	299.04	45.95	41.66	28.67
LD15_2	314.01	160.90	25.94	25.47
LD15_3	324.79	440.25	158.58	74.16
LD16_1	322.05	350.77	25.06	13.53
LD16_2	338.70	2726.22	267.66	75.34
LD16_3	353.87	2403.94	53.75	35.42
Average		342.82	39.15	19.50

instance sets, for LD and VLD in particular, despite the difference of the CPU speed. CVTSP1' can also solve 2 instances that could not be solved by RBA.

To explore the computational reach of CVTSP1', we have generated larger instances using the same generation scheme, for $|T| \in \{16, 17, 18, 19, 20, 25, 30, 35, 40, 45, 50\}$. These instances will be made available by the first author upon request. Table 11 presents the results for $|T| \in \{16, 17, 18, 19, 20\}$, where a CPU time limit of 7200 was used. CVTSP1 and CVTSP1' have successfully solved 59 out of 60 instances, extending the computational reach of exact algorithms from

Table 9 Comparison of the models for VLD instances

Instance	Gambella RBA UB	Gambella MISOC CPU	CVTSP1 CPU	CVTSP1' CPU
VLD11.1	257.24	0.31	6.45	3.67
VLD11.2	324.75	0.08	4.31	4.36
VLD11.3	226.13	0.14	6.36	4.34
VLD12.1	326.04	0.14	3.52	4.69
VLD12.2	274.19	0.21	3.64	4.44
VLD12.3	281.94	0.50	11.17	4.81
VLD13.1	316.71	4.26	28.38	16.13
VLD13.2	239.26	1.56	8.56	8.08
VLD13.3	281.33	0.98	6.88	7.61
VLD14.1	319.63	11.69	20.64	26.55
VLD14.2	300.17	2.38	13.63	10.14
VLD14.3	280.37	3.79	20.56	10.5
VLD15.1	295.33	2.18	20.81	15.61
VLD15.2	314.70	50.91	21.13	14.89
VLD15.3	264.95	5.16	15.59	13.63
VLD16.1	379.91	624.76	46.44	35.06
VLD16.2	355.42	861.46	88.13	62.16
VLD16.3	305.99	3600.00	229.26	157.83
Average		287.25	30.86	22.47

Table 10 Comparison of the average results of the RBA of Gambella et al. (2018) and CVTSP1'

Instance set	Gambella RBA		CVTSP1'	
	CPU	Optimal	CPU	Optimal
SD	660.77	16/18	576.86	17/18
MD	551.42	17/18	246.19	18/18
LD	78.13	18/18	19.50	18/18
VLD	52.66	18/18	22.47	18/18

$|T| = 15$ to $|T| = 20$. It can be observed from Table 11 that the lower bounds provided by CVTSP1' are significantly stronger than those of the MISOC model of Gambella et al. (2018). CVTSP1' outperforms CVTSP1 for 49 out of 60 instances in terms of CPU time. Unfortunately, we cannot provide a comparison with RBA due to the lack of the source code of the authors' implementation.

5.2. Iterated Local Search

We have then proceeded to test the performance of the ILS algorithm, by testing it on the same instance set performing 10 experiments per instance. Tables 12, 13, 14, and 15 report the average results for the instances with $|T| \in \{10, 11, 12, 13, 14, 15, 20\}$, for which we have strong lower bounds provided by CVTSP1'. The average CPU time requirement per instance and the average CPU requirement by instance size are reported under column headings 'CPU' and 'Average CPU', respectively. The average optimality gap, computed as the ratio of the difference between the upper and lower bounds divided by the lower bound, is observed to have an overall average of 0.18%. The only exception is the instance SD15.3, for which none of the three models or RBA could provide an exact solution. The performance is observed to be particularly high for the instance set VLD,

Table 11 Comparison of the models for the new instances with $|T| \in \{16, 17, 18, 19, 20\}$

Instance	Gambella MISOC				CVTSP1				CVTSP1'			
	UB	LB	Gap (%)	CPU	UB	LB	Gap (%)	CPU	UB	LB	Gap (%)	CPU
SD17.1	295.33	234.76	25.80	7200.00	285.58	285.58	0.00	44.07	285.58	285.58	0.00	36.98
SD17.2	357.74	204.54	74.90	7200.00	341.49	341.49	0.00	215.62	341.49	341.49	0.00	190.38
SD17.3	286.28	155.62	83.96	7200.00	270.31	270.31	0.00	123.38	270.31	270.31	0.00	107.42
SD18.1	351.79	222.06	58.42	7200.00	329.47	329.47	0.00	95.45	329.47	329.47	0.00	104.69
SD18.2	338.97	206.07	64.49	7200.00	314.49	314.49	0.00	72.70	314.49	314.49	0.00	58.15
SD18.3	381.83	179.94	112.20	7200.00	341.80	341.80	0.00	397.79	341.80	341.80	0.00	262.13
SD19.1	390.30	191.75	103.55	7200.00	359.70	359.70	0.00	1567.84	359.70	359.70	0.00	1327.38
SD19.2	372.80	179.37	107.84	7200.00	312.15	312.15	0.00	64.26	312.15	312.15	0.00	51.94
SD19.3	388.31	212.70	82.57	7200.00	379.31	379.31	0.00	830.60	379.31	379.31	0.00	429.47
SD20.1	396.96	166.23	138.80	7200.00	365.93	365.93	0.00	2845.95	365.93	365.93	0.00	2317.80
SD20.2	366.12	223.80	63.59	7200.00	355.71	355.71	0.00	1029.80	355.71	355.71	0.00	2857.17
SD20.3	431.74	144.93	197.90	7200.00	341.76	341.76	0.00	694.78	341.76	341.76	0.00	403.03
SD21.1	434.67	209.44	107.54	7200.00	371.82	371.82	0.00	157.74	371.82	371.82	0.00	78.72
SD21.2	419.55	133.77	213.65	7200.00	373.80	373.80	0.00	4217.53	373.80	373.80	0.00	2723.54
SD21.3	350.28	102.39	242.11	7200.00	350.28	350.28	0.00	3471.80	350.28	350.28	0.00	3850.27
Average			111.82	7200.00			0.00	1055.29			0.00	986.60
MD17.1	340.54	243.18	40.04	7200.00	322.21	322.21	0.00	32.13	322.21	322.21	0.00	44.80
MD17.2	375.15	272.96	37.44	7200.00	350.08	350.08	0.00	189.15	350.08	350.08	0.00	91.71
MD17.3	405.54	200.93	101.83	7200.00	356.27	356.27	0.00	39.80	356.27	356.27	0.00	33.61
MD18.1	379.83	211.10	79.93	7200.00	346.43	346.43	0.00	176.01	346.43	346.43	0.00	122.22
MD18.2	341.80	217.00	57.52	7200.00	335.27	335.27	0.00	63.81	335.27	335.27	0.00	60.85
MD18.3	439.54	213.39	105.98	7200.00	376.27	376.27	0.00	117.60	376.27	376.27	0.00	107.19
MD19.1	426.68	146.58	191.09	7200.00	364.45	364.45	0.00	329.67	364.45	364.45	0.00	157.89
MD19.2	424.95	197.06	115.64	7200.00	370.01	370.01	0.00	216.08	370.01	370.01	0.00	119.73
MD19.3	424.84	158.78	167.56	7200.00	345.01	345.01	0.00	114.86	345.01	345.01	0.00	82.63
MD20.1	421.84	174.40	141.89	7200.00	341.91	341.91	0.00	498.29	341.91	341.91	0.00	249.25
MD20.2	359.41	153.24	134.54	7200.00	332.47	332.47	0.00	633.07	332.47	332.47	0.00	676.15
MD20.3	399.58	176.75	126.07	7200.00	342.91	342.91	0.00	191.52	342.91	342.91	0.00	128.94
MD21.1	451.89	109.98	310.88	7200.00	363.66	363.66	0.00	1075.81	363.66	363.66	0.00	792.71
MD21.2	418.01	114.14	266.23	7200.00	344.70	344.70	0.00	1632.04	344.70	344.70	0.00	463.34
MD21.3	455.66	86.31	427.95	7200.00	363.71	363.71	0.00	466.48	363.71	363.71	0.00	442.70
Average			153.64	7200.00			0.00	385.09			0.00	238.25
LD17.1	391.63	217.67	79.92	7200.00	380.11	380.11	0.00	178.32	380.11	380.11	0.00	125.73
LD17.2	358.52	234.40	52.95	7200.00	323.68	323.68	0.00	59.74	323.68	323.68	0.00	46.76
LD17.3	391.46	155.23	152.18	7200.00	349.20	349.20	0.00	103.31	349.20	349.20	0.00	56.40
LD18.1	394.76	164.37	140.16	7200.00	329.43	329.43	0.00	35.29	329.43	329.43	0.00	27.52
LD18.2	342.50	142.67	140.06	7200.00	287.78	287.78	0.00	63.96	287.78	287.78	0.00	92.67
LD18.3	389.83	188.46	106.85	7200.00	368.89	368.89	0.00	124.69	368.89	368.89	0.00	160.26
LD19.1	348.81	166.71	109.23	7200.00	313.50	313.50	0.00	778.58	313.50	313.50	0.00	520.81
LD19.2	384.38	157.53	144.01	7200.00	359.53	359.53	0.00	125.51	359.53	359.53	0.00	141.38
LD19.3	338.75	133.64	153.48	7200.00	323.57	323.57	0.00	1290.56	323.57	323.57	0.00	647.55
LD20.1	407.15	152.59	166.82	7200.00	376.88	376.88	0.00	990.95	376.88	376.88	0.00	521.94
LD20.2	389.09	191.69	102.98	7200.00	334.67	334.67	0.00	1549.34	334.67	334.67	0.00	1630.62
LD20.3	382.88	149.82	155.56	7200.00	334.96	334.96	0.00	2330.24	334.97	334.96	0.00	1299.39
LD21.1	495.10	177.07	179.61	7200.00	400.73	400.73	0.00	534.71	400.73	400.73	0.00	321.13
LD21.2	408.63	134.42	203.99	7200.00	348.67	339.25	2.78	7200.00	348.67	337.64	3.27	7200.00
LD21.3	463.03	118.78	289.81	7200.00	365.98	365.98	0.00	1754.99	365.98	365.98	0.00	1434.91
Average			145.17	7200.00			0.19	1141.35			0.22	948.47
VLD17.1	363.22	261.95	38.66	7200.00	337.94	337.94	0.00	108.42	337.94	337.94	0.00	94.53
VLD17.2	342.21	204.69	67.18	7200.00	303.03	303.03	0.00	31.74	303.03	303.03	0.00	30.62
VLD17.3	398.66	253.57	57.22	7200.00	371.44	371.44	0.00	36.58	371.44	371.44	0.00	32.65
VLD18.1	359.39	228.17	57.51	7200.00	326.79	326.79	0.00	44.47	326.79	326.79	0.00	25.95
VLD18.2	369.31	204.31	80.76	7200.00	344.19	344.19	0.00	412.83	344.19	344.19	0.00	389.31
VLD18.3	358.17	185.15	93.45	7200.00	320.31	320.31	0.00	60.54	320.31	320.31	0.00	40.90
VLD19.1	399.65	190.05	110.29	7200.00	337.27	337.27	0.00	113.66	337.27	337.27	0.00	68.31
VLD19.2	399.02	138.14	188.85	7200.00	314.04	314.04	0.00	71.67	314.04	314.04	0.00	64.92
VLD19.3	425.96	187.38	127.33	7200.00	392.18	392.18	0.00	974.66	392.18	392.18	0.00	559.74
VLD20.1	387.35	115.88	234.28	7200.00	362.99	362.99	0.00	875.97	362.99	362.99	0.00	576.73
VLD20.2	435.12	154.86	180.97	7200.00	355.56	355.56	0.00	163.72	355.56	355.56	0.00	133.79
VLD20.3	381.21	92.50	312.13	7200.00	310.86	310.86	0.00	348.91	310.86	310.86	0.00	168.59
VLD21.1	466.43	115.97	302.19	7200.00	388.97	388.97	0.00	1442.15	388.97	388.97	0.00	3804.03
VLD21.2	505.43	144.73	249.22	7200.00	387.86	387.86	0.00	1342.39	387.86	387.86	0.00	1224.05
VLD21.3	419.35	113.95	268.03	7200.00	337.64	337.64	0.00	4076.81	337.64	337.64	0.00	1944.37
Average			157.87	7200.00			0.00	673.63			0.00	610.57

for which the target points are more sparsely distributed. The ILS algorithm finds the optimal solution for 77% of these instances, in 10 runs out of 10 with different random number seeds.

For larger instances with $|T| \in \{25, 30, 35, 40, 45, 50\}$, we have computed the deviation of the results found by the ILS algorithm from the Best Known Solution (BKS) value to observe the robustness of the performance, the details of which are presented in Tables 16, 17, 18, and 19. The overall deviation is computed as 0.79%, quite uniformly distributed among the four instance sets.

Table 12 Results of the ILS algorithm for instance set SD

Instance	Average UB	LB	Gap (%)	CPU	Average CPU
SD11_1	108.76	108.76	0.00	9.11	
SD11_2	113.92	113.92	0.00	7.95	
SD11_3	125.19	125.19	0.00	9.24	8.76
SD12_1	107.60	107.60	0.00	12.44	
SD12_2	154.02	153.94	0.05	12.25	
SD12_3	118.61	118.61	0.00	12.47	12.38
SD13_1	116.12	116.12	0.00	17.61	
SD13_2	137.06	136.86	0.15	14.29	
SD13_3	121.47	121.47	0.00	17.04	16.32
SD14_1	128.14	128.14	0.00	21.65	
SD14_2	124.66	124.66	0.00	19.45	
SD14_3	138.07	138.07	0.00	20.05	20.38
SD15_1	125.79	123.67	1.71	24.03	
SD15_2	136.64	136.10	0.39	25.18	
SD15_3	134.37	118.36	13.53	23.38	24.20
SD16_1	145.10	145.10	0.00	32.21	
SD16_2	155.58	155.35	0.15	30.37	
SD16_3	128.38	125.70	2.13	31.21	31.26
SD17_1	285.66	285.58	0.03	36.03	
SD17_2	341.49	341.49	0.00	40.97	
SD17_3	270.31	270.31	0.00	37.01	38.00
SD18_1	329.47	329.47	0.00	46.34	
SD18_2	314.49	314.49	0.00	44.48	
SD18_3	342.03	341.80	0.07	44.04	44.95
SD19_1	360.02	359.70	0.09	48.37	
SD19_2	312.15	312.15	0.00	54.76	
SD19_3	379.31	379.31	0.00	52.82	51.98
SD20_1	365.93	365.93	0.00	56.73	
SD20_2	355.92	355.71	0.06	53.10	
SD20_3	341.76	341.76	0.00	62.59	57.47
SD21_1	371.82	371.82	0.00	71.05	
SD21_2	375.85	373.80	0.55	66.34	
SD21_3	350.93	350.28	0.19	65.70	67.69

5.3. Experiments on the instances of Poikonen and Golden (2019)

We have compared the models with the branch-and-bound algorithm of Poikonen and Golden (2019) on the instances generated by the authors. These instances are distinctly different in nature, where the speed of the vehicle is only twice that of the carrier i.e. $V_v = 2$, but the autonomy of

Table 13 Results of the ILS algorithm for instance set MD

Instance	Average UB	LB	Gap (%)	CPU	Average CPU
MD11_1	146.85	146.85	0.00	9.49	
MD11_2	132.12	132.12	0.00	9.59	
MD11_3	133.48	133.42	0.05	9.18	9.42
MD12_1	157.85	157.74	0.07	13.31	
MD12_2	165.74	165.63	0.06	13.51	
MD12_3	121.24	121.24	0.00	12.45	13.09
MD13_1	151.11	150.89	0.15	15.54	
MD13_2	131.35	130.99	0.27	16.53	
MD13_3	150.37	150.37	0.00	16.10	16.06
MD14_1	146.95	146.95	0.00	19.52	
MD14_2	163.59	163.59	0.00	18.82	
MD14_3	153.01	153.01	0.00	19.44	19.26
MD15_1	168.24	168.24	0.00	23.17	
MD15_2	136.94	136.94	0.00	23.15	
MD15_3	157.86	157.86	0.00	23.72	23.35
MD16_1	166.21	166.21	0.00	34.78	
MD16_2	177.50	177.23	0.15	30.46	
MD16_3	164.37	164.37	0.00	29.75	31.66
MD17_1	322.21	322.21	0.00	38.20	
MD17_2	350.08	350.08	0.00	36.55	
MD17_3	356.27	356.27	0.00	40.59	38.45
MD18_1	346.47	346.43	0.01	46.71	
MD18_2	335.27	335.27	0.00	42.63	
MD18_3	376.41	376.27	0.04	45.62	44.99
MD19_1	364.45	364.45	0.00	48.23	
MD19_2	370.01	370.01	0.00	51.52	
MD19_3	345.01	345.01	0.00	48.04	49.26
MD20_1	341.91	341.91	0.00	59.70	
MD20_2	332.49	332.47	0.00	54.35	
MD20_3	342.91	342.91	0.00	60.96	58.34
MD21_1	363.77	363.66	0.03	69.91	
MD21_2	344.71	344.70	0.00	69.76	
MD21_3	363.71	363.71	0.00	74.47	71.38

the vehicle is $a = 20$. For these instances, both the MISOC of Gambella et al. (2018) and the branch-and bound algorithm of Poikonen and Golden (2019) can outperform CVTSP1', as the results in Table 20 show. Each row of Table 20 corresponds to the average result of 25 instances for the corresponding instance type and size. We think that there are two possible reasons for the degradation of the performance of CVTSP1' on these instances. First, the sets (35) - (37) are potentially smaller, thereby reducing the number of optimality cuts in Proposition 5. Second, the big- M constraints (17), (19), and (21) in CVTSP1' are considerably weaker due to the large value of a , which presumably leads to looser relaxations. Notably, (48) improved the performance of CVTSP1' for these instances.

Finally, we have observed that ILS performs well for this set of instances with known optimal solutions, with an average optimality gap of 0.37% for the regular instances and 1.40% for the clustered instances.

Table 14 Results of the ILS algorithm for instance set LD

Instance	Average UB	LB	Gap (%)	CPU	Average CPU
LD11_1	311.55	311.55	0.00	10.62	
LD11_2	345.20	345.19	0.00	10.15	
LD11_3	299.53	299.53	0.00	10.05	10.27
LD12_1	296.07	296.07	0.00	12.55	
LD12_2	308.26	308.26	0.00	12.59	
LD12_3	270.31	270.31	0.00	14.08	13.07
LD13_1	261.64	261.64	0.00	17.24	
LD13_2	294.85	294.85	0.00	16.99	
LD13_3	307.34	307.34	0.00	17.32	17.19
LD14_1	319.80	319.79	0.00	21.41	
LD14_2	282.92	282.91	0.00	19.64	
LD14_3	301.60	301.60	0.00	22.22	21.09
LD15_1	299.04	299.04	0.00	26.32	
LD15_2	314.01	314.01	0.00	24.94	
LD15_3	324.79	324.79	0.00	25.70	25.65
LD16_1	322.05	322.05	0.00	31.75	
LD16_2	338.70	338.70	0.00	31.20	
LD16_3	354.89	353.87	0.29	29.38	30.77
LD17_1	380.11	380.11	0.00	38.45	
LD17_2	323.68	323.68	0.00	37.38	
LD17_3	349.20	349.20	0.00	36.23	37.36
LD18_1	329.43	329.43	0.00	45.54	
LD18_2	287.78	287.78	0.00	41.00	
LD18_3	369.28	368.89	0.10	40.96	42.50
LD19_1	313.64	313.50	0.04	52.77	
LD19_2	359.53	359.53	0.00	51.09	
LD19_3	323.57	323.57	0.00	47.94	50.60
LD20_1	376.88	376.88	0.00	57.20	
LD20_2	334.67	334.67	0.00	58.10	
LD20_3	334.96	334.96	0.00	60.60	58.63
LD21_1	400.73	400.73	0.00	78.02	
LD21_2	349.17	339.25	2.92	72.73	
LD21_3	366.11	365.98	0.04	66.89	72.55

6. Conclusions

We have studied the CVTSP and provided structural properties that pertain to optimal solutions, i.e., perfect synchronization of the Carrier and the Vehicle, the maximum feasible take-off and landing distances for the Vehicle at the target points, and the conditions under which the Vehicle would utilize all of its autonomy in an optimal solution. We have presented a new mixed integer second order conic optimization model, based on the properties and subtour elimination constraints from the Traveling Salesman Problem literature, and augmented it with optimality cuts. Extensive computational experiments have demonstrated the superiority of our model and that it is capable of solving instances with up to 20 target points. We have also provided an ILS algorithm that can provide near-optimal solutions within 10 minutes of CPU time for instances with up to 50 target points.

Table 15 Results of the ILS algorithm for instance set VLD

Instance	Average UB	LB	Gap (%)	CPU	Average CPU
VLD11_1	257.24	257.24	0.00	10.51	
VLD11_2	324.75	324.75	0.00	11.13	
VLD11_3	226.13	226.13	0.00	10.92	10.86
VLD12_1	326.04	326.04	0.00	14.59	
VLD12_2	274.19	274.19	0.00	15.67	
VLD12_3	281.94	281.94	0.00	13.48	14.58
VLD13_1	316.74	316.71	0.01	19.59	
VLD13_2	239.26	239.26	0.00	18.62	
VLD13_3	281.33	281.33	0.00	18.24	18.81
VLD14_1	319.63	319.63	0.00	23.04	
VLD14_2	300.17	300.17	0.00	25.53	
VLD14_3	280.37	280.37	0.00	25.28	24.62
VLD15_1	295.33	295.33	0.00	28.47	
VLD15_2	314.70	314.70	0.00	30.90	
VLD15_3	264.95	264.95	0.00	29.80	29.72
VLD16_1	379.91	379.91	0.00	31.04	
VLD16_2	355.42	355.42	0.00	35.41	
VLD16_3	306.32	305.99	0.11	31.14	32.53
VLD17_1	337.94	337.94	0.00	39.93	
VLD17_2	303.03	303.03	0.00	39.06	
VLD17_3	371.44	371.44	0.00	41.58	40.19
VLD18_1	326.80	326.79	0.00	45.35	
VLD18_2	344.79	344.19	0.17	44.39	
VLD18_3	320.31	320.31	0.00	50.47	46.73
VLD19_1	337.27	337.27	0.00	51.07	
VLD19_2	314.04	314.04	0.00	51.60	
VLD19_3	392.18	392.18	0.00	50.39	51.02
VLD20_1	362.99	362.99	0.00	57.22	
VLD20_2	355.60	355.56	0.01	59.94	
VLD20_3	310.86	310.86	0.00	57.39	58.18
VLD21_1	389.40	388.97	0.11	63.77	
VLD21_2	387.86	387.86	0.00	72.35	
VLD21_3	337.64	337.64	0.00	63.90	66.68

Acknowledgments

We thank Claudio Gambella for providing the benchmark problem instances. This study was partially supported by University of Bath International Research Funding Scheme. This support is gratefully acknowledged. Finally, we thank the three anonymous referees for their constructive comments, which have improved the paper considerably.

Table 16 Results of the ILS algorithm for larger SD instances

Instance	Average UB	BKS	Deviation (%)	CPU	Average CPU
SD26_1	393.56	393.56	0.00	114.25	
SD26_2	413.12	413.12	0.00	109.32	
SD26_3	378.69	378.69	0.00	112.36	111.98
SD31_1	408.99	408.99	0.00	186.34	
SD31_2	352.59	352.59	0.00	157.93	
SD31_3	390.10	388.68	0.36	161.96	168.74
SD36_1	479.52	479.52	0.00	240.47	
SD36_2	413.77	413.77	0.00	234.43	
SD36_3	384.90	384.90	0.00	246.15	240.35
SD41_1	485.61	485.61	0.00	328.54	
SD41_2	452.43	452.43	0.00	329.00	
SD41_3	455.08	455.08	0.00	365.14	340.89
SD46_1	504.77	504.77	0.00	460.09	
SD46_2	417.48	417.48	0.00	456.20	
SD46_3	509.55	509.55	0.00	466.12	460.80
SD51_1	449.58	449.58	0.00	611.47	
SD51_2	448.58	448.58	0.00	620.98	
SD51_3	512.18	512.18	0.00	631.85	621.43

Table 17 Results of the ILS algorithm for larger MD instances

Instance	Average UB	BKS	Deviation (%)	CPU	Average CPU
MD26_1	345.22	345.22	0.00	122.42	
MD26_2	417.27	416.73	0.13	104.45	
MD26_3	413.35	412.09	0.31	109.62	112.16
MD31_1	424.24	424.24	0.00	173.22	
MD31_2	413.55	413.47	0.02	171.95	
MD31_3	473.10	473.10	0.00	172.87	172.68
MD36_1	431.14	431.14	0.00	244.57	
MD36_2	419.99	419.99	0.00	246.84	
MD36_3	375.21	375.21	0.00	225.14	238.85
MD41_1	448.76	448.76	0.00	313.00	
MD41_2	427.43	427.43	0.00	326.27	
MD41_3	474.80	474.80	0.00	338.76	326.01
MD46_1	475.46	475.46	0.00	447.51	
MD46_2	485.85	485.85	0.00	442.42	
MD46_3	527.79	527.79	0.00	449.39	446.44
MD51_1	471.55	471.55	0.00	648.69	
MD51_2	498.48	498.48	0.00	602.42	
MD51_3	509.48	504.62	0.96	631.86	627.66

Table 18 Results of the ILS algorithm for larger LD instances

Instance	Average UB	BKS	Deviation (%)	CPU	Average CPU
LD26_1	363.06	362.36	0.19	116.08	
LD26_2	387.36	387.36	0.00	111.56	
LD26_3	371.37	369.81	0.42	109.64	112.43
LD31_1	412.46	412.32	0.03	158.96	
LD31_2	382.81	382.81	0.00	162.85	
LD31_3	426.00	426.00	0.00	165.45	162.42
LD36_1	447.29	447.29	0.00	233.80	
LD36_2	436.95	436.95	0.00	240.79	
LD36_3	473.32	473.32	0.00	231.24	235.28
LD41_1	460.80	460.80	0.00	329.50	
LD41_2	482.70	482.70	0.00	327.44	
LD41_3	491.53	491.53	0.00	343.40	333.45
LD46_1	483.54	483.54	0.00	442.82	
LD46_2	488.92	488.92	0.00	451.89	
LD46_3	541.70	541.70	0.00	455.37	450.03
LD51_1	504.69	504.69	0.00	606.34	
LD51_2	498.95	498.95	0.00	672.29	
LD51_3	493.20	493.20	0.00	650.29	642.97

Table 19 Results of the ILS algorithm for larger VLD instances

Instance	Average UB	BKS	Deviation (%)	CPU	Average CPU
VLD26_1	344.97	344.97	0.00	106.89	
VLD26_2	417.36	415.45	0.46	110.17	
VLD26_3	386.05	386.05	0.00	109.15	108.73
VLD31_1	407.16	407.16	0.00	166.33	
VLD31_2	407.27	407.27	0.00	170.40	
VLD31_3	401.43	401.43	0.00	181.80	172.84
VLD36_1	421.04	421.04	0.00	226.01	
VLD36_2	415.35	415.35	0.00	232.88	
VLD36_3	407.88	406.44	0.35	234.82	231.24
VLD41_1	460.33	459.25	0.23	331.93	
VLD41_2	420.32	420.32	0.00	338.05	
VLD41_3	450.69	450.69	0.00	323.29	331.09
VLD46_1	425.48	425.48	0.00	446.74	
VLD46_2	490.50	490.50	0.00	440.85	
VLD46_3	473.92	473.92	0.00	477.84	455.14
VLD51_1	494.95	494.95	0.00	626.69	
VLD51_2	464.93	464.93	0.00	662.18	
VLD51_3	494.45	494.45	0.00	640.56	643.14

Table 20 Computational results for the instances of Poikonen and Golden (2019)

Instance type	Instance size	Poikonen branch-and-bound	Gambella MISOC		CVTSP1'	
		CPU	CPU	Gap (%)	CPU	Gap (%)
Regular	10	1.54	54.56	0.00	1724.13	0.71
	15	18.80	5897.45	7.78	7200.00	19.46
	20	700.22	7200.00	48.13	N/A	N/A
Clustered	10	12.59	31.04	0.00	2713.42	0.00
	15	559.56	3939.31	1.35	7200.00	22.45
	20	N/A	7200.00	16.10	N/A	N/A

Appendix

We now provide the proofs of Propositions 3 and 4.

A. Proof of Proposition 3

Proof. Assume that the conditions (i) – (iii) hold for some target point $j \in T$, but the Vehicle does not use its full autonomy at target point j in any optimal solution. By Proposition 1, we can assume that the Carrier and the Vehicle are perfectly synchronized at all target points. Suppose that $s_{to,j}$ and $s_{l,j}$ are defined as in Table 2. Proposition 2 and the contrapositive of Corollary 3 imply that

$$\max \{ \|s_{to,j} - q_j\|, \|s_{l,j} - q_j\| \} < \frac{a(V_v + V_c)}{2}.$$

It follows by (i) and (iii) that $s_{to,j} \neq p_o$ and $s_{l,j} \neq p_f$, respectively. Furthermore, we claim that there does not exist a target point $j' \in T \setminus \{j\}$ such that $s_{to,j} = s_{l,j'}$ or $s_{l,j} = s_{to,j'}$. Suppose, for a contradiction, that $s_{to,j} = s_{l,j'}$ for some $j' \in T \setminus \{j\}$. Then, by Proposition 2, we should have

$$\|s_{to,j} - q_{j'}\| = \|s_{l,j'} - q_{j'}\| \leq \frac{a(V_v + V_c)}{2}.$$

Then, by the triangle inequality,

$$\|q_j - q_{j'}\| \leq \|q_j - s_{to,j}\| + \|s_{to,j} - q_{j'}\| < \frac{a(V_v + V_c)}{2} + \frac{a(V_v + V_c)}{2} = a(V_v + V_c), \quad (49)$$

which contradicts the condition (ii). A similar argument can be employed if $s_{l,j} = s_{to,j'}$ for some $j' \in T \setminus \{j\}$.

We therefore make the following claim: If the Vehicle does not use full autonomy at a target point j that satisfies the conditions (i) – (iii), then the solution can be modified so that the Vehicle uses full autonomy without compromising optimality.

Suppose that target point j satisfies the conditions (i) – (iii) and, for simplicity, let us denote the take-off and landing points by to and ℓ , respectively. Let us denote the point before to and the point after ℓ on the trajectory of the Carrier by i and k , respectively. Consider a circle of radius $a(V_v + V_c)/2$ centered at target point j . In Figures 4 and 5, the blue solid lines represent the trajectory of the Carrier whereas the blue dashed lines correspond to that of the Vehicle. By Corollary 3, both to and ℓ should be strictly inside this circle whereas neither of the points i and k can lie strictly inside this circle by the previous argument.

There are two cases:

Case 1: Suppose that the points i , to , ℓ , and k are all collinear (see Figure 4). In this case, suppose that to is replaced by to' located on the boundary of the circle and ℓ is replaced by ℓ' , which is located at a distance of $V_c a$ from to' so that the travel time of the Carrier is exactly equal to a (see the first illustration in Figure 4). Denoting the coordinates of to' and ℓ' by $s_{to',j}$ and $s_{\ell',j}$, respectively, the triangle inequality yields

$$\|q_j - s_{\ell',j}\| \geq \|s_{to',j} - q_j\| - \|s_{to',j} - s_{\ell',j}\| = \frac{a(V_v + V_c)}{2} - V_c a = \frac{a(V_v - V_c)}{2},$$

which implies that the travel time of the Vehicle from to' to q_j and from q_j to ℓ' satisfies

$$\frac{\|s_{to',j} - q_j\| + \|q_j - s_{\ell',j}\|}{V_v} \geq \frac{a(V_v + V_c) + a(V_v - V_c)}{2V_v} = a.$$

Therefore, if the travel time of the Vehicle is equal to a , then we obtain an optimal solution with perfect synchronization since the mission completion time remains the same and the Vehicle uses its full autonomy, which establishes our claim.

If the travel time of the Vehicle is strictly greater than a , then we move the pair to' and ℓ' towards the inside of the circle while maintaining the distance of $V_c a$ between them. Consider the case in which the triangle with the base given by the line segment between to' and ℓ' and the apex given by j forms an isosceles triangle (see the second illustration in Figure 4). By Lemma 2, the height of the triangle with the base given by the line segment between to and ℓ and the apex given by j satisfies

$$h_j \leq \frac{t_j^*}{2} \sqrt{V_v^2 - V_c^2} < \frac{a}{2} \sqrt{V_v^2 - V_c^2},$$

where the last inequality is due to our assumption that the Vehicle does not use its full autonomy. Clearly, the triangle with the base given by the line segment between to' and ℓ' and the apex given by j also has height h_j . Denoting the travel time of the Vehicle from to' to j and from j to ℓ' by t , we obtain

$$\frac{V_v^2 t^2}{4} = h_j^2 + \frac{V_c^2 a^2}{4} < \frac{a^2(V_v^2 - V_c^2)}{4} + \frac{V_c^2 a^2}{4} = \frac{V_v^2 a^2}{4}$$

by the Pythagorean theorem, which implies that $t < a$. Since the travel time of the Vehicle changes continuously as to' moves away from the boundary of the circle, we conclude that there exists a pair of points to^* and ℓ^* such that the travel time of each of the Carrier and the Vehicle is equal to a . Therefore, we obtain an optimal solution with perfect synchronization since the mission completion time remains the same and the Vehicle uses its full autonomy.

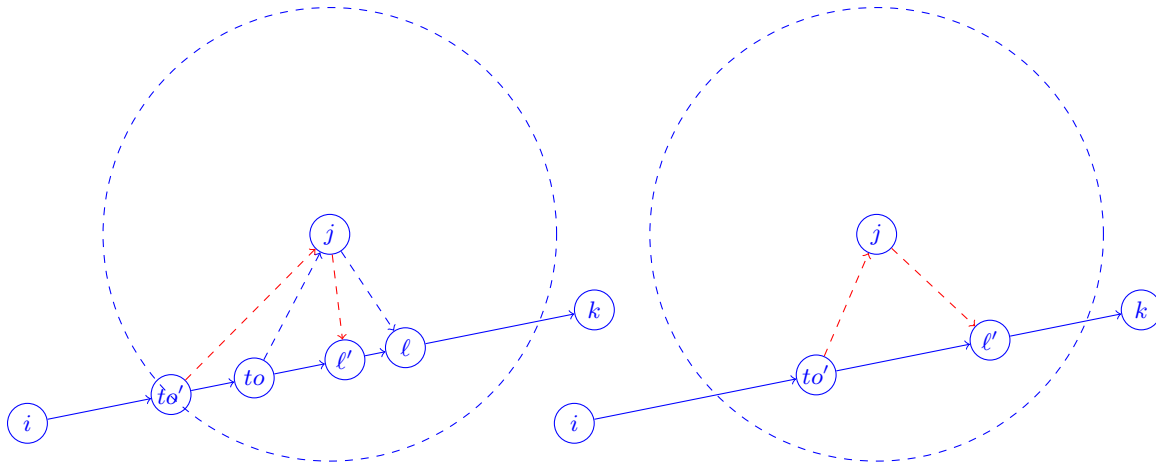


Figure 4 Illustration of Case 1 in the proof of Proposition 3

Case 2: Suppose that the points i , to , ℓ , and k are not collinear. In this case, we claim that the travel time of the Carrier between to and ℓ is equal to a (i.e., $t_j^c = a$) in every optimal solution. Suppose, for a contradiction, that there exists an optimal solution such that $t_j^c < a$. By Lemma 1, we obtain $t_j^v \leq t_j^c < a$. There are two subcases:

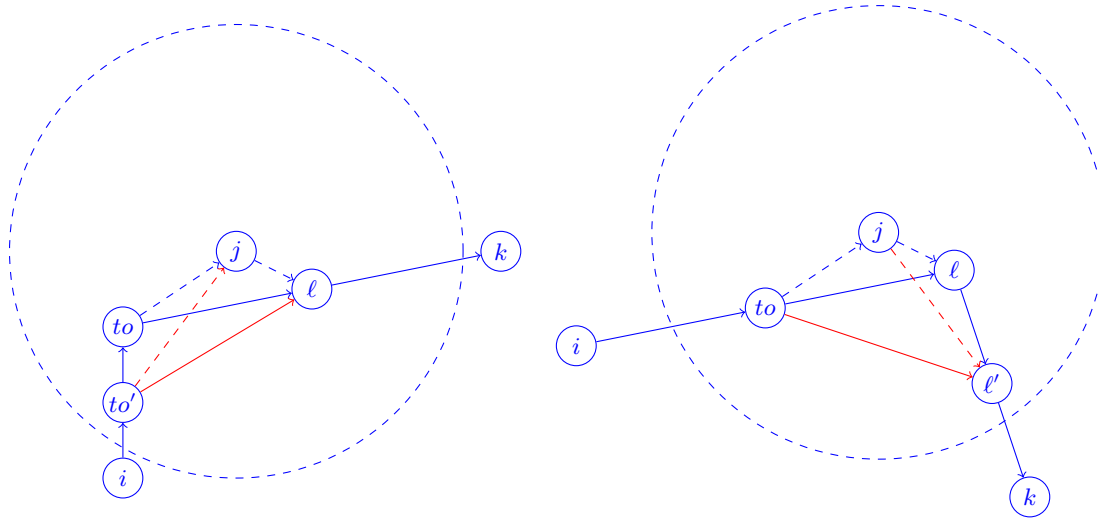


Figure 5 Illustration of Case 2 in the proof of Proposition 3

Case 2a: Suppose that i , to , and ℓ are not collinear. Then, we can replace to by a point to' on the incoming path of the Carrier which is closer to the boundary of the circle (see the first illustration in Figure 5) while ensuring that the maximum of the travel time of the Vehicle and the Carrier is less than or equal to a . Therefore, we retain feasibility of the solution. By the triangle inequality, the travel time of the Carrier between i and ℓ is strictly improved since i , to , and ℓ are not collinear, which contradicts the optimality of the original solution. Therefore, the travel time of the Carrier between to and ℓ is equal to a in every optimal solution. By Proposition 2, there exists an optimal solution with perfect synchronization, i.e., there exists an optimal solution such that $t_v^j = t_c^j = a$. The assertion follows.

Case 2b: Suppose that to , ℓ , and k are not collinear. Then, we can replace ℓ by a point ℓ' on the outgoing path of the Carrier which is closer to the boundary of the circle (see the second illustration in Figure 5). A similar argument as in Case 2a yields a contradiction. By invoking Proposition 2 once again, we complete the proof. $\square$

B. Proof of Proposition 4

Proof. We follow a similar argument as in the proof of Proposition 3. Under the assumption that either (i) or (ii) is satisfied and either (iii) and (iv) is satisfied, we can adjust the take-off point to and ℓ , if necessary (see Figures 4 and 5), so that the Vehicle uses its full autonomy without worsening the objective function value. $\square$

References

- N.A.H. Agatz, P. Bouman, and M. Schmidt. Optimization approaches for the traveling salesman problem with drone. *Transportation Science*, 52(4):965–981, 2018.
- E. M. Arkin and R. Hassin. Approximation algorithms for the geometric covering salesman problem. *Discrete Applied Mathematics*, 55(3):197–218, 1994.

- M. Battarra, G. Erdoğan, G. Laporte, and D. Vigo. The traveling salesman problem with pickups, deliveries, and handling costs. *Transportation Science*, 44(3):383–399, 2010.
- H. L. Bodlaender, C. Feremans, Al. Grigoriev, E. Penninkx, R. Sitters, and T. Wolle. On the minimum corridor connection problem and other generalized geometric problems. *Computational Geometry*, 42(9):939–951, 2009.
- P. Bouman, N.A.H. Agatz, and M. Schmidt. Dynamic programming approaches for the traveling salesman problem with drone. *Networks*, 72(4):528–542, 2018.
- T.-H. Hubert Chan and Shaofeng H.-C. Jiang. Reducing curse of dimensionality: Improved PTAS for TSP (with neighborhoods) in doubling metrics. In *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2016, Arlington, VA, USA, January 10-12, 2016*, pages 754–765, 2016.
- W.P. Coutinho, R.Q. do Nascimento, A.A. Pessoa, and A. Subramanian. A branch-and-bound algorithm for the close-enough traveling salesman problem. *INFORMS Journal on Computing*, 28(4):752–765, 2016.
- M. de Berg, J. Gudmundsson, M. J. Katz, C. Levkopoulos, M. H. Overmars, and A. F. van der Stappen. TSP with neighborhoods of varying size. *Journal of Algorithms*, 57(1):22–36, 2005.
- A. Dumitrescu and J. S. B. Mitchell. Approximation algorithms for TSP with neighborhoods in the plane. *Journal of Algorithms*, 48(1):135–159, 2003.
- C. Gambella, A. Lodi, and D. Vigo. Exact solutions for the carrier-vehicle traveling salesman problem. *Transportation Science*, 52(2):320–330, 2018.
- E. Garone, R. Naldi, A. Casavola, and E. Frazzoli. Cooperative path planning for a class of carrier-vehicle systems. In *Proceedings of the 47th IEEE Conference on Decision and Control, CDC 2008, December 9-11, 2008, Cancún, Mexico*, pages 2456–2462, 2008.
- E. Garone, R. Naldi, A. Casavola, and E. Frazzoli. Planning algorithms for a class of heterogeneous multi-vehicle systems. In *Proceedings of the 8th IFAC Symposium on Nonlinear Control Systems, September 1-3, 2010, University of Bologna, Italy*, pages 969–974, 2010a.
- E. Garone, R. Naldi, A. Casavola, and E. Frazzoli. Cooperative mission planning for a class of carrier-vehicle systems. In *Proceedings of the 49th IEEE Conference on Decision and Control, CDC 2010, December 15-17, 2010, Atlanta, Georgia, USA*, pages 1354–1359, 2010b.
- E. Garone, R. Naldi, and A. Casavola. Traveling salesman problem for a class of carrier-vehicle systems. *Journal of Guidance, Control, and Dynamics*, 34(4):1272–1276, 2011.
- E. Garone, J.-F. Determe, and R. Naldi. Generalized traveling salesman problem for carrier-vehicle systems. *Journal of Guidance, Control, and Dynamics*, 37(3):776–774, 2014.
- C. Groër, B. Golden, and E. Wasil. A library of local search heuristics for the vehicle routing problem. *Mathematical Programming Computation*, 2(2):79–101, 2010.

- D.J. Gulczynski, J.W. Heath, and C.C. Price. The close enough traveling salesman problem: A discussion of several heuristics. In F.B. Alt, M.C. Fu, and B. L. Golden, editors, *Perspectives in Operations Research: Papers in Honor of Saul Gass' 80th Birthday*, pages 271–283. Springer US, Boston, MA, 2006.
- Q.M. Ha, Y. Deville, Q.D. Pham, and M.H. Ha. On the min-cost traveling salesman problem with drone. *Transportation Research Part C: Emerging Technologies*, 86:597–621, 2018.
- M. Klauc, S. Blazek, M. Kvasnica, and M. Fikar. Mixed-integer SOCP formulation of the path planning problem for heterogeneous multi-vehicle systems. In *European Control Conference, ECC 2014, Strasbourg, France, June 24-27, 2014*, pages 1474–1479, 2014.
- J. S. B. Mitchell. A PTAS for TSP with neighborhoods among fat regions in the plane. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2007, New Orleans, Louisiana, USA, January 7-9, 2007*, pages 11–18, 2007.
- C.C. Murray and A.G. Chu. The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery. *Transportation Research Part C: Emerging Technologies*, 54:86–109, 2015.
- C.C. Murray and R. Raj. The multiple flying sidekicks traveling salesman problem: Parcel delivery with multiple drones. *Transportation Research Part C: Emerging Technologies*, 110:368–398, 2020.
- R. M. Murray. Recent research in cooperative control of multivehicle systems. *Journal of Dynamic Systems, Measurement, and Control*, 129(5):571–583, 2007.
- S. Poikonen and B. Golden. The mothership and drone routing problem. *INFORMS Journal on Computing*, 2019.
- S. Poikonen, B. L. Golden, and E. A. Wasil. A branch-and-bound approach to the traveling salesman problem with a drone. *INFORMS Journal on Computing*, 31(2):335–346, 2019.
- S. Safra and O. Schwartz. On the complexity of approximating TSP with neighborhoods and related problems. *Computational Complexity*, 14(4):281–307, 2006.
- R. G. M. Saleu, L. Deroussi, D. Feillet, N. Grangeon, and A. Quilliot. An iterative two-step heuristic for the parallel drone scheduling traveling salesman problem. *Networks*, 72(4):459–474, 2018.
- A. Subramanian, E. Uchoa, and L.S. Ochi. A hybrid algorithm for a class of vehicle routing problems. *Computers & Operations Research*, 40(10):2519–2531, 2013.
- UNEP. After the tsunami: Rapid environmental assessment, 2005. URL <http://wedocs.unep.org/handle/20.500.11822/8372>.